0.4 C++-Konzepte

0.4.1 Operatoren in C++

0.4.1.1 Operatorbegriff

Beispiel: Addition, Operator +, Operanden a, b

- Infix-Notation: $a + b$
  - konventionelle mathematische Darstellung eines arithmetischen Ausdrucks
  - Auswertungsreihenfolge abhängig von Prioritäten der Operatoren und Klammerung
- Präfix-Notation: $+ab$
  - Operator steht vor den beiden Operanden
  - „polnische Notation“ (von polnischem Mathematiker Lukasiewicz)
  - *keine* Abhängigkeit von Prioritäten und Klammerung
  - Verwendung z. B. in Lisp
- Postfix-Notation: $ab+$
  - Operator steht hinter den beiden Operanden
  - „umgekehrte polnische Notation“
  - automatisierte Auswertung mittels ADT Stack (Stapel) (vgl. 0.3.1.2) leicht möglich

0.4.1.2 Auswertung eines Postfixausdrucks

Auswertung des nächsten Zeichens:

- Wenn Operand, dann auf Stack ablegen *(push)*
- Wenn *n*-stelliger Operator, dann
  - oberste *n* Elemente vom Stack nehmen *(pop)*
  - Operator auf diese *n* Elemente anwenden
  - Ergebnis auf den Stack ablegen *(push)*

© R.-R. Grigat 2003-2019

Beispiel

3 5 + 6 4 − ∗

Step	1	2	3	4	5	6	7	8
						4		
			5		6	6	2	
Stack		3	3	8	8	8	8	16

0.4.1.3 Unäre, binäre und ternäre Operatoren

Definition: Unärer (einstelliger, monadischer) Operator

- Unärer Operator hat *einen* Operanden
- Beispiel Negierungsoperator: `a = -b;`

Definition: binärer (zweistelliger, dyadischer) Operator

- Binärer Operator hat *zwei* Operanden
- Beispiel Additionsoperator: `c = a + b;`

Definition: Ternärer (dreistelliger) Operator

- Dreistelliger Operator hat *drei* Operanden
- In C++ hat nur ?-Operator drei Operanden
- `a?b:c;` bedeutet: *Wenn a, dann b, sonst c*

0.4.1.4 Vorrangtabelle für Operatoren

Vorrangtabelle für Operatoren
Unäre Operatoren und Zuweisungsoperatoren sind *rechtsassoziativ* (Suffix bindet stärker als Präfix), alle anderen *linksassoziativ* (Auswertung von links nach rechts).

1. Klammerung (ausdr), Lambda-Ausdrücke
2. klassenname::member, namespace::member, ::name

© R.-R. Grigat 2003-2019

3. objekt.member, zeiger->member
4. Suffixoperatoren("von rechts"): Indizierung zeiger[ausdr], Funktionsaufruf ausdr(ausdr-liste), Postinkrement L-Wert++, Postdekrement L-Wert––, Cast-Operatoren static / dynamic / reinterpret / const, Funktionsartige Typumwandlung typ(ausdr-liste)
5. Präfixoperatoren("von links"): Präinkrement ++L-Wert, Prädekrement ––L-Wert, Komplement ~, nicht !, unäres +, unäres –, Adresse &, Dereferenzierung *, new, delete, noexcept, Cast (typ)ausdr
6. Arithmetik *, /,
7. Arithmetik +, -
8. Schieben «, »
9. Vergleiche <, <=, >=, >
10. Vergleiche ==, !=
11. diverse Stufen logischer Operatoren
12. Ternärer Operator ausdr ? ausdr : ausdr
13. Zuweisungen =, *=, /=, +=, -=,

0.4.1.5 Beispiele Vorrang

Beispiele Vorrang

- `a=b=c` bedeutet `a=(b=c)`
- `a+b+c` bedeutet `(a+b)+c`
- `a<<b<<c` bedeutet `(a<<b)<<c`
- `*p++` bedeutet `*(p++)`
- `int*p[]` bedeutet Array aus Zeigern auf `int`
- `int(*p)[]` bedeutet Zeiger auf Array p aus `int`
- `int*p()` bedeutet Funktion p, die Zeiger auf `int` liefert

Beispiel: OperatorVerkettung, KP12_4_operatorenMitReferenztyp

© R.-R. Grigat 2003-2019

0.4.1.6 Operator vs. Funktion

Beispiel Addition

- Infix-Notation: `a + b`
- Funktionsschreibweise für globale Funktion: `plus(a,b)`
  - n-stelliger Operator kann als *Funktion* mit n Parametern interpretiert werden
- Funktionsschreibweise für Memberfunktion: `a.plus(b)`
  - n-stelliger Operator kann als *Memberfunktion* mit $n-1$ Parametern interpretiert werden

0.4.1.7 Additionsoperation bei eigenen Klassen: Additionsfunktion

Option 1: Additionsfunktion

```
1 //ccomplex.hpp
2 class CComplex
3 {
4   public:
5     CComplex plus(CComplex &other);
6     /* ... */
7 }
```

```
1 //ccomplex.cpp
2 CComplex CComplex::plus(CComplex &other)
3 {
4     return CComplex(re()+other.re(),
5                     im()+other.im());
6 }
```

```
1 // main.cpp
2 /* ... */
3   CComplex a(0.0,1.0);
4   CComplex b(1.0,0.0);
5   CComplex c = a.plus(b);  // unschön, + wäre besser
```

© R.-R. Grigat 2003-2019

0.4.1.8 Additionsoperation bei eigenen Klassen: Überladen

Option 2: Überladen des Operators +

```
1 //ccomplex.hpp
2 class CComplex
3 {
4   public:
5     CComplex operator+(CComplex &other);//Überladen der Methode operator+
6     /* ... */                          //die bei Operation + aufgerufen wird
7 }                                       //a + b bewirkt a.operator+(b)
```

```
1 //ccomplex.cpp
2 CComplex CComplex::operator+(CComplex &other)
3 {
4     return CComplex(re()+other.re(),
5                     im()+other.im());
6 }
```

```
1 // main.cpp
2 /* ... */
3   CComplex a(0.0,1.0);
4   CComplex b(1.0,0.0);
5   CComplex c = a + b;  // gewohnte Schreibweise, nun auch für CComplex
```

0.4.1.9 Überladen von Operatoren in C++

- Definition eines Operators für eigenen Datentypen
- Programmcode wird übersichtlich und leichter verständlich
  - falls Operatoren *sinnvoll* überladen werden
  - z. B. keine Subtraktion mit + implementieren!
- Operator überladen als *Memberfunktion* einer Klasse
  - erster Operand ist immer das Objekt, aus dem Operator aufgerufen wird
  - `a + b` bewirkt `a.operator+(b)`
- Alternativ globale Funktion mit zwei Aufrufparametern definieren, wie z.B. `A& operator+(A&,A&)`
- C++ Funktionsattribute (`static`, `virtual`, ...) auch auf Operatoren anwendbar

Beispiel: KP19_1_operator_daytime, KP19_2_operator_euro1, KP19_3_globalerOperatorFriend_euro

© R.-R. Grigat 2003-2019

0.4.1.10 Überladen von operator+=

Beispiel: Überladen von **operator+=**

```
1 //ccomplex.hpp
2 class CComplex
3 {
4 /* ... */
5   CComplex& operator+=
6     (CComplex& z);
7 };
```

```
1 //ccomplex.cpp
2 CComplex& CComplex::operator
3   +=(CComplex& z)
4 {
5   m_re += z.re();
6   m_im += z.im();
7   return *this;
8 }
```

```
1 //main.cpp
2   CComplex a(1.0,2.0);
3   CComplex b(3.0,4.0);
4   a += b;
```

Anmerkungen

- *Operator ist* Memberfunktion `CComplex::operator+=`
- *Leerzeichen wäre zulässig*, d. h. `CComplex::operator +=`
- *erster Operand ist* aktuelles Objekt (`a` in `main.cpp`)
- `a += b` *bewirkt* `a.operator+=(b)`

0.4.1.11 Überladen von operator[]

Überladen von operator[]

- Der Indexoperator [] bewirkt den Aufruf der Memberfunktion **operator[]**.

© R.-R. Grigat 2003-2019

- Realisierung mehrdimensionaler Felder mittels Komposition von Klassen
- Beispiel Matrizen als Spaltenobjekte, die Zeilenobjekte enthalten.

Beispiel: KP30_4_dynamischeMatrizen

0.4.2 Ein- und Ausgabe mit C++-Streams

0.4.2.1 Einleitung

- In C++ werden „Datenkanäle“ mit *Streams* beschrieben.
- Beispiele:
  - Datenkanal zu einer Datei auf der Festplatte
  - Datenkanal zu einem zweiten Rechner per Internetverbindung
  - Datenkanal zum Drucker
  - Datenkanal für Bildschirmausgabe/Tastatureingabe
- Programmierer können eigene Streams zur Verfügung stellen
- Der C++-Standard stellt für Datei-I/O und Benutzer-I/O (Bildschirmausgabe/Tastatureingabe) vordefinierte Streams zur Verfügung

0.4.2.2 Ein- und Ausgabe mit Streams

Ausgabe mit Streams

```
1 //main.cpp
2 #include<iostream>
3 using namespace std;
4 /* ... */
5 int max_kmph = 50;
6
7 cout << "Innerhalb geschlossener Ortschaften darf"
8      << max_kmph << " km/h gefahren werden." << endl;
```

Eingabe mit Streams

© R.-R. Grigat 2003-2019

```
1 //main.cpp
2 #include<iostream>
3 using namespace std;
4 /* ... */
5 int alter;
6
7 cout << "Bitte geben Sie Ihr Alter ein: ";
8 cin  >> alter;
```

0.4.2.3 Vordefinierte Streams in C++

- Drei vordefinierte Stream-*Instanzen* für Benutzer-I/O:
  - `cout`: Standardausgabe: Wird auf Bildschirm ausgegeben
  - `cin` : Standardeingabe: Wird von Tastatur eingelesen
  - `cerr`, `clog`: Standard-Fehlerausgabe
    - für Fehlermeldungen
    - wird auf Bildschirm ausgegeben
    - kann aber auch umgelenkt werden, z. B. in log-Dateien
- Vordefinierte Stream-*Klasse* für Dateien:
  - `fstream`:
    - Instantiierung mit Dateinamen
    - Datei wird im Konstruktor geöffnet
- Vordefinierte Stream-*Klasse* für Zeichenketten:
  - `stringstream`: Erlaubt die Ausgabe von Daten in eine Zeichenkette

0.4.2.4 Manipulatoren und Methoden zur Formatierung

- Mit Streams ist *formatiertes* Ausgeben von Daten möglich.
- Erreicht wird das Formatieren mittels
  - Manipulatoren, die in den Stream eingefügt werden
  - Memberfunktionen der Stream-Objekte
  - Header `<iomanip>`

© R.-R. Grigat 2003-2019

Beispiel

```
double pi=3.14159264;

cout << "1. Pi = " << pi << endl;  //  1. Pi = 3.14159

cout.precision(2); // alternativ cout << setprecision(2)
cout << "2. Pi = " << fixed << pi << endl;  // 2. Pi = 3.14

cout << "3. Pi = " << scientific << pi << endl;
// Ausgabe: 3. Pi = 3.14e+000
```

0.4.2.5 Relevante Manipulatoren

Manipulator	Aufgabe
std::oct	Oktale Ausgabe
std::hex	Hexadezimale Ausgabe
std::dec	Dezimale Ausgabe (Standard)
std::showpos	Positive Zahlen mit Vorzeichen
std::noshowpos	Positive Zahlen ohne Vorzeichen
std::fixed	Festkommadarstellung
std::scientific	Gleitkommadarstellung
std::setprecision(int n)	n Nachkommastellen
std::setw(int n)	Feldbreite n Zeichen
std::setfill(int ch)	Füllzeichen ch
std::left	linksbündig
std::right	rechtsbündig
std::internal	Vorzeichen links, Wert rechts

Beispiel: KP04_1_oct_dec_hex, KP04_2_showpointFixedScientific, KP04_3_cinGetline, KP04_4_AusgabeManipulator, KP04_5_cinHexGleitpunkt, KP04_6_cinGetGetline

0.4.2.6 Kopieren einer Textdatei in C++

Beispiel

```
#include <string>
#include <fstream>
#include <cstdlib>
using namespace std;

```

© R.-R. Grigat 2003-2019

```
6  int main()
7  {
8    ifstream in("main.cpp");
9    ofstream out("copy.cpp");
10   string s;
11   while(getline(in, s))
12   {
13     out << s << "\n";
14   }
15   return EXIT_SUCCESS;
16 }
```

- **ifstream** öffnet Datei zum Lesen
- **ofstream** öffnet Datei zum Schreiben
- **getline** liest Zeichenkette bis Zeilenende (inklusive)
  - Ausgabe exklusive Zeilenende
  - Zeilenende \n muss explizit in Kopie eingefügt werden
- **getline** liefert **false** bei Dateiende

0.4.3 Arrays

Arrays (Felder, Vektoren)

- Struktur zur Speicherung (nahezu) beliebiger Anzahl gleicher Datentypen
- Feld a der Größe n: Elemente $a_0 \ldots a_{n-1}$

Beispiel in C++

```
1 //main.cpp
2
3 // Feld fuer Temperaturdaten von 100 Messungen
4 float data[100];
5
6 data[0]  = 10.0;   // Zugriff auf den ersten Wert
7 data[3]  = 20.3;   // Zugriff auf den vierten Wert
8 data[99] = -4.8;   // Zugriff auf den letzten Wert
```

Beispiel: KP16_1_array, KP16_2_array_fibonacci, KP17_1_textPtr, KP17_2_arrayelement_adresse KP17_3_kleiner_sync, KP17_4_reverse, KP17_5_suche_strstrEigeneVersion

© R.-R. Grigat 2003-2019

0.4.3.1 Ablage von Feldern im Speicher

Beispiel: `float data[100];`

- `data` Adresse des ersten Elements
- `data+n` Adresse des (n+1)-ten Elements
- `*(data+n)` Inhalt des (n+1)-ten Elements
- `data[n]` ≡ `*(data+n)`

Adresse	Name	Inhalt
0x3f60	???	???
0x3f64	`data[0]`	10.0
0x3f68	`data[1]`	16.3
...	...	...
0x40f0	`data[99]`	-4.8
0x40f4	`data[100]`	???

Tabelle 4: Speicheraufbau bei Feldern

Achtung:

- `data+99+n` Adresse n Elemente hinter letztem Element
- *Keine Fehlermeldung* des Compilers
- Ergebnis nicht vorhersagbar

0.4.3.2 Signatur der main-Funktion

main-Funktion

```
1 int main(int argc, char * argv[]){
2 ....// Funktionsblock
3 }
```

Parameter der main-Funktion

- argv ist ein Array aus Zeigern auf char.

© R.-R. Grigat 2003-2019

- Die Namen argc und argv sind frei wählbar, aber üblich.
- argc Anzahl Argumente, mindestens 1
- argv[0] zeigt auf den Programmnamen einschl. Pfad.
- argv[1] zeigt auf das erste eigentliche Argument
- argv[argc-1] zeigt auf das letzte Argument
- argv[argc] ist `nullptr`, der Null-Zeiger

Beispiel: KP17_6_kommandozeile_main

0.4.3.3 Felder im automatischen Speicher

Problem der Felder im automatischen Speicher

- Größe des Feldes muss zur Kompilierzeit festgelegt sein
- Benötigter Speicherbedarf muss dazu bereits bekannt sein
- Was tun, wenn Feldgröße erst zur Laufzeit bekannt wird?

nicht erlaubt:

```
1 int n=10;
2 float data[n]; // Fehler: n nicht const!
```

Lösung

- Feld im dynamischen Speicher erzeugen
- Empfehlung: STL-Container benutzen (s. 0.5.6)
- Hier als Alternative: Feld selbst erzeugen

© R.-R. Grigat 2003-2019

0.4.3.4 Felder im dynamischen Speicher

- Reservierung des benötigten Speicherplatzes für *n* Elemente mit **new** [n]
- geschieht *zur Laufzeit*
- muss mit **delete**[] gelöscht werden (einschl. [])
  - **delete** ohne [] löscht nur erstes Element
  - Ursache von Speicherlecks

Beispiel

```
1 int n=100;
2
3 // Dynamische Reservierung eines Feldes
4 float *data = new float[n];
5 data[10] = -4.5;
6 // Freigabe des Speicherplatzes
7 delete[] data;
```

Beispiel: KP21_3_dynamisch_Arrays

© R.-R. Grigat 2003-2019

0.4.4 Kopierkonstruktor, Zuweisungsoperator, Move-Semantik

0.4.4.1 Wiederholung: Konstruktoren

- Werden beim Erzeugen von Objektinstanzen aufgerufen (vgl. 0.3.2.11)
- Können mehrfach (bei unterschiedlichen Signaturen) existieren
  - Stichwort: Überladen von Funktionen (vgl. 0.2.4.9)

Beispiel

```
1  //ccomplex.hpp
2  class CComplex
3  {
4  public:
5     CComplex();
6           // parameterloser Konstruktor
7     CComplex(double re, double im);
8           // Konstruktor mit 2 Parametern
9  private:
10    double m_re;
11    double m_im;
12 };
```

0.4.4.2 Kopierkonstruktor

Merkmale des Kopierkonstruktors

- Konstruktor mit eigener Klasse als Parameter
- Initialisiert neues Objekt mit Werten eines bereits vorhandenen Objektes derselben Klasse
- Compiler stellt Default-Kopierkonstruktor zur Verfügung
  - kopiert alle Membervariablen
    - Membervariablen der Kopie werden mit den Werten der Membervariablen des Originalobjektes initialisiert
    - Es wird Kopierkonstruktor jedes einzelnen Memberobjekts aufgerufen
  - Ausnahme: statische Variablen, diese existieren nur einmal im Speicher
  - Existiert nur, wenn kein eigener Kopierkonstruktor definiert wird

© R.-R. Grigat 2003-2019

Beispiel: Nachbau des default-Kopierkonstruktors

```
1 //ccomplex.hpp
2 class CComplex
3 {
4 public:
5   CComplex();
6   //parameterloser Konstruktor
7   CComplex(const CComplex & other);
8   //Deklaration des Kopierkonstruktors
9 /* ... */
10 private:
11   double m_re;
12   double m_im;
13   static bool polar;
14 }
```

```
1 //ccomplex.cpp
2 //Definition des Kopierkonstruktors
3 CComplex::CComplex
4       (const CComplex & other):
5 m_re(other.m_re), //Initialisierung der
6 m_im(other.m_im)  //Membervariablen
7 {
8 }
```

```
1 // main.cpp
2
3 CComplex a;     // parameterloser Konstruktor
4 CComplex b(a);   // Kopierkonstruktor
```

0.4.4.3 Notwendigkeit eines Kopierkonstruktors

Eigene Kopierkonstruktoren notwendig bei

- Objekten mit Zeigern als Membervariablen
  - wenn auch der Speicherbereich, auf den die Zeiger zeigen, dupliziert werden soll
- speziell gewünschtem Verhalten, z. B. Verändern von statischen Membervariablen

© R.-R. Grigat 2003-2019

0.4.4.4 Beispiel eines Kopierkonstruktors

Beispiel C++

```
//ccomplexpair.hpp

class CComplexPair{
public:
  CComplexPair(const CComplexPair & other);
  //[..]
private:
  CComplex *m_pFirst;
  CComplex *m_pSecond;
};

CComplexPair::CComplexPair(const CComplexPair & other):
  m_pFirst ( new CComplex(*(other.m_pFirst )) ),
  m_pSecond( new CComplex(*(other.m_pSecond)) )
{}
//Kopierkonstruktor von CComplex, jeweils mit new aufgerufen
//Achtung: delete in Destruktor nicht vergessen!
```

```
//main.cpp
CComplexPair b(a); // b als Kopie von a erzeugt
```

0.4.4.5 Kopierkonstruktor und Zeiger

Beispiel

```
//main.cpp
CComplexPair b(a); // b als Kopie von a erzeugt
```

Verhalten *ohne* eigenen Kopierkonstruktor

- Verwendung von default-Kopierkonstruktor
- Zeiger von verschiedenen Instanzen zeigen auf *denselben* Speicherbereich
- Veränderung von Objekt *a* wirkt sich direkt auf Objekt *b* aus
- Ursache von vielen Fehlern

Beispiel: CComplexPair

© R.-R. Grigat 2003-2019

0.4.4.6 Implementierung von speziellem Verhalten in Kopierkonstruktor

Beispiel C++

```
1 //schaf.hpp
2
3 class Schaf{
4 public:
5   Schaf();
6   Schaf(const Schaf & other);
7 private:
8   double m_gewicht;
9   static int m_anzahlSchafe;
10 };
```

```
1 //schaf.cpp
2
3 Schaf::Schaf(const Schaf & other):
4 m_gewicht(other.m_gewicht)
5 {
6   ++m_anzahlSchafe;
7   //Motivation für speziellen
8   //Kopierkonstruktor
9 }
```

m_anzahlSchafe

- speichert Gesamtzahl aller existierender Schafe
- statische Variable
- muss bei Erzeugen einer Kopie erhöht werden

© R.-R. Grigat 2003-2019

0.4.4.7 Beispiele zu Initialisierung und Zuweisung

```
1 //Testclass.hpp
2 class Testclass
3 {
4   public:
5     Testclass();
6     Testclass(const Testclass& other);
7     Testclass(const int value);
8     ~Testclass();
9     Testclass& operator=
10        (const Testclass& other);
11 };
```

```
1 //main.cpp
2 Testclass a;
3 Testclass b(a);
4 Testclass c = 3; //Initialisierung
5 Testclass d = Testclass(3);
6 Testclass(3);
7 Testclass e = Testclass(a);
8 a = 3; //nur hier Zuweisung!
```

Erläuterungen `main.cpp`

- Symbol = je nach Kontext *Initialisierung (Zeilen 4,5,7) oder Zuweisung (Zeile 8)*
- Überladung von **`operator=`** betrifft *nur* Zuweisung, also Auswirkung nur in Zeile 8.
- Initialisierungsoperator = kann *nicht* überladen werden
- *Zeile 2:* Parameterloser Konstruktor
- *Zeile 3:* Kopierkonstruktor
- *Zeile 4:*
  - *Initialisierung* über Aufruf des Konstruktors mit **`int`** Parameter
  - Identisch zu `Testclass c(3);`
- *Zeile 5:*
  - *Initialisierung* über Aufruf des Konstruktors mit **`int`** Parameter

© R.-R. Grigat 2003-2019

  - Identisch zu `Testclass d(3);`

- Zeile 6:
  - Temporäres namenloses Objekt
  - Aufruf des Konstruktors mit **`int`** Parameter
  - Objekt wird nach dieser Zeile sofort wieder zerstört
  - Anwendung: Objekt als Parameter einer Funktion

- Zeile 7:
  - *Initialisierung* über Aufruf des Kopierkonstruktors
  - Identisch zu `Testclass e(a);`

- Zeile 8:
  1. Aufruf des Konstruktors mit **`int`** Parameter (temporäres namenloses Objekt)
  2. Keine Initialisierung, sondern Aufruf des Zuweisungsoperators **`operator=`**
  3. Zerstörung des temporären Objekts

- Nur falls in `Testclass` der **`operator=(const int& other)`**, d. h. mit passender **`int`**-Signatur definiert wäre:
  - In Zeile 8 Aufruf des Zuweisungsoperators **`operator=`** mit **`int`**-Parameter
  - *Kein* Konstruktoraufruf!

- Jede Klasse erhält bei Bedarf *Default*-Zuweisungsoperator mit demselben Klassentyp als Parameter

© R.-R. Grigat 2003-2019

0.4.4.8 Überladen des Zuweisungsoperators

Beispiel: Zuweisungsoperator

```
1  //ccomplexpair.hpp
2
3  class CComplexPair{
4  public:
5    CComplexPair & operator=
6        (const CComplexPair & other);
7  /* ... */
8  private:
9    CComplex *m_pFirst;
10   CComplex *m_pSecond;
11 };
```

```
1  //ccomplexpair.cpp
2  CComplexPair & CComplexPair::operator=
3      (const CComplexPair & other)
4  {
5    *m_pFirst  = *(other.m_pFirst);
6    *m_pSecond = *(other.m_pSecond);
7    return *this;
8  }
```

Anmerkungen

- Referenz auf `*this`, also eigenes Objekt, zurückliefern
- `a=b=c;` bewirkt Ausführung von `a=(b=c);`
  - Funktionsaufruf ist `a.operator=(b.operator=(c))`
  - `a`, `b` und `c` erhalten gleichen Inhalt, aber separaten Speicher

0.4.4.9 Kopierkonstruktor vs. Zuweisungsoperator

Kopierkonstruktor

- Mit `new` Speicher reservieren
- Kein Rückgabewert in Konstruktoren!
- Initialisierung aller Membervariablen!

© R.-R. Grigat 2003-2019

```
CComplexPair::CComplexPair
    (const CComplexPair & other)
{
  m_pFirst  =
      new CComplex(*(other.m_pFirst ));
  m_pSecond =
      new CComplex(*(other.m_pSecond));
}
```

Zuweisungsoperator

- Keinen zusätzlichen Speicher reservieren
- Keine Initialisierungen vornehmen

```
CComplexPair & CComplexPair::operator=
    (const CComplexPair & other)
{
  *m_pFirst
      = *(other.m_pFirst);
  *m_pSecond
      = *(other.m_pSecond);
  return *this;
}
```

0.4.4.10 L-Werte und R-Werte

L-Wert (L-value)

- L-Wert ist ein Ausdruck, der auf einen *zusammenhängenden Speicherbereich* verweist (mit oder ohne Namen, z.B. mit **new** erzeugt)
- L-Werte sind modifizierbar oder konstant
- *Konstanter* L-Wert kann nicht links von Zuweisung stehen
- Beispiel für L-Wert: Dereferenzierung `*p[a+10]`

R-Wert (R-value)

- Ausdruck, der kein L-Wert ist

© R.-R. Grigat 2003-2019

- Beispiele: temporäre Werte, a+10
- erzeugbar aus L-Wert lVar mit `std::move(lVar)`
  - identisch zu `static_cast<X&&>(lVar)`, lVar hat Typ X
  - Name `std::move()` historisch, kein move sondern cast

0.4.4.11 move() und forward() in <utility>

move()

- `move()` ist Typumwandlung in einen R-Wert.
- Man hätte `move()` besser `rvalue()` nennen sollen.

```
template<typename T>
Remove_reference<T>&& move(T&& t) noexcept {
return static_cast<Remove_reference<T>&&>(t);  }
```

0.4.4.12 move() und forward() in <utility> (2)

forward()

- `forward()` ist ebenfalls Typumwandlung in einen R-Wert, aber für L-Wert und auch für R-Wert.
- `forward()` gilt als perfekte Weiterleitung eines Arguments von einer Funktion zu einer anderen.
- `forward()` ist sicher, während `move()` das Objekt für eine Zerstörung markiert.

```
template<typename T>
T&& forward(Remove_reference<T>&  t) noexcept {
return static_cast<T&&>(t); }
template<typename T>
T&& forward(Remove_reference<T>&& t) noexcept {
static_assert(!Is_lvalue_reference<T>,"L-Wert weiterleiten");
return static_cast<T&&>(t); }

int i = 7;
forward<int>(i); // L-Wert gelangt zur ersten  Version
forward<int>(7); // R-Wert gelangt zur zweiten Version
```

© R.-R. Grigat 2003-2019

0.4.4.13 Identität und Verschiebbarkeit

Identität und Verschiebbarkeit

- Objekte mit *Identität* (GL-Wert) als Name, Zeiger oder Referenz lassen sich auf Gleichheit oder Änderung prüfen
  - GL-Wert (generalized lvalue) besitzt Identität
  - PR-Wert (purer R-Wert) besitzt keine Identität
- *Verschiebbare* Objekte (R-Werte) besitzen dynamische Inhalte, die verschoben werden können und Originalobjekt in gültigem, aber nicht spezifiziertem Zustand hinterlassen

	GL-Wert *hat Identität*	**PR-Wert** *keine Identität*
R-Wert *verschiebbar*	**X-Wert** `std::move(L-Wert)`	purer R-Wert
nicht verschiebbar	**L-Wert** modifizierbar oder const	—

0.4.4.14 Verschieben versus tiefer Kopie

Verschieben versus tiefer Kopie

- Bei Grundtypen und Objekten ohne Zeiger wird stets kopiert.
- Bei Objekten mit Zeigern unterschiedliches Verhalten (das selbst zu implementieren ist)
  - *Kopie* erzeugt *tiefe* Kopie.
  - *Verschieben* (Move-Semantik) erzeugt *flache* Kopie, setzt dann die im Original enthaltenen Zeiger auf `nullptr`.
  - Bei Verschieben wird Original verändert, daher nur für R-Werte zulässig.
  - Move-Semantik ist (bei großen dynamischen Datenstrukturen) viel effizienter als Kopie.
- `nullptr` kann nur Zeigern zugewiesen werden, keine Verwechslung mit Ganzzahlen `NULL` oder `0`.

© R.-R. Grigat 2003-2019

0.4.4.15 R-Wert-Referenzen in Signatur

R-Wert-Referenzen && in Signatur von Funktionen

- *R-Wert-Referenz* verweist auf temporäres, *veränderbares* Objekt unter *Annahme keiner erneuten Verwendung*
- Für Grundtypen und Objekte *ohne* Zeiger verhält sich R-Wert-Referenz && identisch zur L-Wert-Referenz &
- Für Grundtypen und Objekte *mit* Zeiger können die Zeiger der R-Wert-Referenz nach Bedarf modifiziert werden
  - Motivation: (nur) dynamische Inhalte temporärer Objekte *verschieben* statt zeitaufwändiger tiefer Kopie
  - *Move-Konstruktor* statt Kopierkonstruktor
  - *Move-Zuweisung* statt Zuweisung

0.4.4.16 Kopierkonstruktur versus Move-Konstruktor

Kopierkonstruktor versus Move-Konstruktor

- Argument von Kopierkonstruktor ist L-Wert-Referenz
- Argument von Move-Konstruktor ist R-Wert-Referenz
- Argument von Move-Konstruktor kann sich ändern, darf nicht const sein

Beispiel

```
1 class CAuto
2 {
3 CAuto(const CAuto&);   // Kopierkonstruktor, L-Wert-Referenz als Parameter
4 CAuto(      CAuto&&);  // Move-Konstruktor, nur R-Wert-Referenz zulässig als
5                        // Parameter, diesen nicht const setzen (kann
6 ...                                    // verändert werden)!
7 }
```

Beispiel: KP22_1_moveSemantik_floatVek

© R.-R. Grigat 2003-2019

0.4.4.17 Move-Konstruktor

Kopierkonstruktor

```
CComplexPair::CComplexPair(const CComplexPair & lVar) // lVar ist L-Wert
{ m_pFirst  = new CComplex(*(lVar.m_pFirst ));
  m_pSecond = new CComplex(*(lVar.m_pSecond));
}
```

Move-Konstruktor

```
CComplexPair::CComplexPair(      CComplexPair && rVar) // rVar ist R-Wert
{ m_pFirst  = rVar.m_pFirst;  // Dynamischen Speicher übernehmen (Move)
  m_pSecond = rVar.m_pSecond;
  rVar.m_pFirst  = nullptr;     // Dyn. Speicher vor Freigabe schützen,
  rVar.m_pSecond = nullptr;     //      daher rVar nicht const übergeben!
}     // nun erfolgt automatischer Aufruf des Destruktors von rVar,
      // dynamischer Speicher ist aber vor Freigabe geschützt
```

```
//main.cpp
CComplexPair a, b;
CComplexPair c(std::move(a+b)); // Move-Konstruktor
```

0.4.4.18 Move-Zuweisung

Vorteile von Move-Zuweisung und -Konstruktor

- keine Neureservierung von dynamischem Speicher
- nur flache Kopie, kein zeitaufwändiger Kopiervorgang
- wird aufgerufen, wenn Argument ein R-Wert ist
- Argument *kann verändert werden*, darf nicht const sein
- konstante Laufzeit

Beispiel MOVE-Zuweisung als Swapper

```
CComplexPair& CComplexPair::operator=(CComplexPair && rVal)
{ if(this != &rVal)    // keine Selbstzuweisung erlauben
  {  CComplex* temp = m_pFirst;    // swap m_pFirst
           m_pFirst  = rVal.m_pFirst;
     rVal.m_pFirst = temp;
```

© R.-R. Grigat 2003-2019

```
6                 temp = m_pSecond;
7             m_pSecond = rVal.m_pSecond;      // swap m_pSecond
8         rVal.m_pSecond = temp;
9     }
10    return *this;
11 }
```

0.4.5 Assertions

0.4.5.1 Hilfsmittel zur Überprüfung von Programmcode

- Software kann Fehler produzieren durch
  - falschen Programmcode
  - fehlerhafte Bedienung
- Fehlertolerante Software
  - erkennt und behandelt
    - fehlerhafte Berechnungen
    - fehlerhafte Daten
  - verhindert undefiniertes Verhalten
- Mögliche Hilfsmittel:
  - Zusicherungen (Assertions)
  - Ausnahmen (Exceptions)

0.4.5.2 Zusicherungen (Assertions)

Definition: Assertion

- Annahme über Beziehung von Variablen zueinander
- Prädikat, d.h. boolescher Ausdruck
- kann Anwesenheit, nicht aber Abwesenheit von Fehlern zeigen
- Checks zur Compilierzeit und zur Laufzeit

mögliche Realisierung in C++

© R.-R. Grigat 2003-2019

```
bool allesGut = true;
/* Code, bei Problemen allesGut=false setzen */
if(!allesGut) exit(-1);
```

0.4.5.3 Typfunktionen (Traits)

Typfunktionen (Traits)

- Zahlreiche Typfunktionen in `<type_traits>`
  - Ermittlung der Eigenschaften von Typen
  - Erzeugung zusammengesetzter Typen
- Überprüfungen zur Übersetzungszeit
- Trait: Merkmal, Eigenschaft, Wesenszug

Kleine Auswahl	Erläuterung
`is_integral<X>`	Ist X integraler Typ?
`is_floating_point<X>`	Ist X Gleitkommatyp?
`is_array<X>`	Ist X ein integriertes Array?
`is_pointer<X>`	Ist X ein Zeiger?
`is_member_function_pointer<X>`	Zeiger auf nicht-statische Memberfunktion?
`is_class<X>`	Ist X **class** oder **struct**?
`is_function<X>`	Ist X eine Funktion?
`is_rvalue_reference<X>`	Ist X eine R-Wert-Referenz?

0.4.5.4 `static_assert()` und `assert()`

zur Compilierzeit: `static_assert()`

```
// Alles prüfen, was in Form konstanter Ausdrücke darstellbar ist,
// hier: Größe von Ganzzahlen prüfen; Meldung, falls Bedingung false
static_assert(4<=sizeof(int), "Ganzzahl zu klein");
template<typename T> void f(T& a) {
static_assert(std::is_floating_point<X>::value,"Gleitkommatyp erwartet");
//...
}
```

zur Laufzeit: `assert()`

© R.-R. Grigat 2003-2019

```
#include <cassert>
bool allesGut = true;
/* ... hier kann allesGut=false gesetzt werden ... */
assert(allesGut);
```

`assert()` in Präprozessor abschaltbar (z. B. im Endprodukt)

- schnellere Programmausführung
- geringere Programmgröße

0.4.6 Ausnahmen

0.4.6.1 Ausnahmen

Ausnahmen (*exceptions*)

- führen wie `assert()` zum Verlassen des ursprünglichen Programmablaufs
- erlauben im Gegensatz zu `assert()` Fehlerbehandlungsroutine für
  - mögliche Fehlerbehebung (z. B. ungültige Eingabe wiederholen)
  - Information für Anwender
  - Vermeidung des Programmabbruchs
- werden durch *Werfen einer Ausnahme* erzeugt
- können von anderen Teilen des Programms *gefangen* werden
- stellen Informationen über die Fehlerursache bereit

0.4.6.2 Werfen einer Ausnahme

Werfen einer Ausnahme in C++

- Das Werfen einer Ausnahme erfolgt mit Hilfe der C++ - Anweisung **throw**
- Parameter des **throw** - Befehls ist eine beliebige Klasse, die Fehlerinformationen beinhaltet

Beispiel: Definition einer Klasse für Ausnahmen

© R.-R. Grigat 2003-2019

```
1  //xausnahme.hpp
2  class XAusnahme
3  {
4  public:
5    XAusnahme
6       (const std::string fehlermeldung);
7    const std::string grund();
8  private:
9    const std::string m_fehlermeldung;
10 };
```

```
1  //xausnahme.cpp
2  XAusnahme::XAusnahme
3       (const std::string fehlermeldung):
4    m_fehlermeldung(fehlermeldung)
5    {}
6  const std::string XAusnahme::grund()
7    {
8      return m_fehlermeldung;
9    }
```

Beispiel: Werfen einer Ausnahme

```
1  //cdatei.cpp, Definition von oeffnen
2  void CDatei::oeffnen()
3  {
4    bool dateiexistiertnicht=false;
5    //[..]
6    if (dateiexistiertnicht)
7    {
8      // Hier wird eine Instanz von XAusnahme geworfen
9      throw XAusnahme("Datei existiert nicht");
10   }
11   /* ... */
12 }
```

0.4.6.3 Fangen einer Ausnahme

Fangen einer Ausnahme in C++

- Wirft man innerhalb `try{Codeblock}` eine Ausnahme, so kann (nur) direkt nach `try{Codeblock}` mit `catch` gefangen werden.

© R.-R. Grigat 2003-2019

- Alle Ausnahmen, die im Codeblock von **try** auftreten, werden gefangen
- Das Programm setzt bei einer geworfenen Ausnahme mit dem **catch**-Befehl fort, ansonsten danach
- Im **catch**-Block kann die Fehlerinformation verarbeitet werden
- Fehlen **try**- und **catch**-Blöcke und wird dennoch eine Ausnahme geworfen, so bricht das Programm mit einer „unbekannten Ausnahme“ ab, denn das Programm ist implizit von einem **try**{Codeblock} umgeben.

Fangen einer Ausnahme

```
1 try {
2   dateioeffnen(); //wirft evtl. XAusnahme
3   /* ... */
4 }
5 catch (XAusnahme &fehler) {
6   cerr << "Ausnahme ist aufgetreten: "
7        << fehler.grund() << endl;
8   throw;  // Dieselbe Ausnahme kann erneut geworfen werden
9 }
```

- Keine Ausnahme auslösen, während andere Ausnahme noch behandelt wird.
- Bei Start von **catch()** gilt Ausnahme als fertig behandelt.
- Innerhalb **catch()** kann dieselbe Ausnahme mit **throw;** erneut geworfen werden, sollte dazu aber in übergeordnetem **try**- Block stehen.

0.4.6.4 Benutzung verschiedener Ausnahmedatentypen

Werfen unterschiedlicher Ausnahmen

- Gezeigtes Beispiel beinhaltete als Fehlerinformation lediglich eine Zeichenkette mit einer Fehlermeldung
- Für verschiedene Fehlertypen können unterschiedliche Informationen notwendig sein
- Lösung: Werfen verschiedener Ausnahmedatentypen

© R.-R. Grigat 2003-2019

0.4.6.5 Abgeleitete Ausnahmedatentypen

Beispiel: Abgeleitete Ausnahmetypen

```
1 //xausnahme.hpp
2 class XAusnahme {
3 public:
4   XAusnahme
5     (const std::string fehlermeldung);
6   const std::string grund();
7 private:
8   const std::string m_fehlermeldung;
9 };
```

```
1 //cdatei.hpp
2 class XDateiGroesse: public XAusnahme
3 {
4 public:
5   XDateiGroesse(int dateigroesse);
6   const int groesse();
7 private:
8   const int m_groesse;
9 };
```

```
1 //datei.cpp, Definition von oeffnen
2 void CDatei::oeffnen()
3 {
4 /* ... */
5 // try-Block siehe main.cpp
6   if (dateiexistiertnicht)
7   {      //Fehlertyp XAusnahme werfen
8     throw XAusnahme
9       ("Datei existiert nicht");
10  }
11  if (dateizugross)
12  {      //Fehlertyp XDateiGroesse werfen
13    throw XDateiGroesse(dateigroesse);
14  }
15 }
```

© R.-R. Grigat 2003-2019

0.4.6.6 Fangen unterschiedlicher Ausnahmedatentypen

Fangen unterschiedlicher Ausnahmetypen

- Ein **try**-Block kann alle Arten von Ausnahmen werfen, unabhängig vom Datentyp des Parameters
- Es wird *nur der erste* **catch**-Block ausgeführt, dessen Parametertyp mit der geworfenen Ausnahme übereinstimmt
- Der Block **catch(...)** fängt alle Datentypen

Beispiel: KP28_1_tryCatch

```
1  //main.cpp
2  try
3  {
4    dateioeffnen();  // Enthaelt throw-Befehle
5  }
6  catch (XDateiGroesse &fehler)
7  {
8    cerr << fehler.grund() << endl;
9    cerr << "Dateigroesse: "
10        << fehler.groesse() << endl;
11 }
12 catch (XAusnahme &fehler)
13 {
14   cerr << "Ausnahme ist aufgetreten: "
15        << fehler.grund() << endl;
16 }
17 catch (...)
18 {
19   cerr << "Nicht speziell behandelte Ausnahme "
20        << "ist aufgetreten" << endl;
21 }
```

0.4.6.7 noexcept

noexcept

- Wirft eine Funktion prinzipiell keine Ausnahmen, so kann dies mit noexcept explizit gemacht werden.

© R.-R. Grigat 2003-2019

- Verständnis des Programms einfacher für Programmierer
- Hilfreich für Optimierung durch Compiler
- Programmstopp durch `std::terminate()`, falls trotzdem eine Ausnahme geworfen wird.
- Bedingtes noexcept ist `noexcept(const bool)` mit Prädikat als Aufrufparameter, bei `noexcept(false)` darf Ausnahme ausgelöst werden, nicht bei `noexcept(true)`.

```
double compute(double) noexcept;  // darf keine Ausnahme auslösen
```

Beispiel: KP28_2_noexcept

0.4.7 Typumwandlungen (casts)

0.4.7.1 Automatische Typumwandlungen in C++

- Der C++ Compiler führt implizite Konvertierung zwischen Ganzzahl- und Fließkommadatentypen automatisch durch.
- Bei Umwandlungen, bei denen Genauigkeit verloren gehen kann (z. B. **`float`** nach **`int`**) *kann* ein (guter) Compiler eine Warnmeldung ausgeben.
- Regeln für zweistellige Operatoren mit Operanden ungleichen Typs:
  1. Bei zwei Gleitkommatypen wird Typ mit der geringeren Genauigkeit in den anderen Typ gewandelt
  2. Ist nur ein Operand Gleitkommatyp, so wird der Ganzzahltyp in diesen Gleitkommatyp gewandelt
  3. Falls beide Operanden Ganzzahltypen sind, wird der Operand mit der geringeren Genauigkeit in den Typ des anderen Operanden gewandelt
  4. Falls ein Operand **`unsigned`** ist, wird der andere Operand ebenfalls nach **`unsigned`** gewandelt

0.4.7.2 Beispiele: Implizite Konvertierung

Beispiele impliziter Konvertierung

© R.-R. Grigat 2003-2019

```
int i = -2;
unsigned int ui = 2;
float f = 0.2;

cout << "i * ui = " << i * ui << endl;
  //Umwandlung von i nach unsigned int
  //Ausgabe: 4294967292

cout << "f * i = " << f * i << endl;
  //Umwandlung von i nach float
  //Ausgabe: -0.4

cout << "i/10*f = " << i/10*f << endl;
  //int-Division i/10, dann Umwandlung des Ergebnisses in float,
  / dann Multiplikation
  //Ausgabe: 0

cout << "i/10.0*f = " << i/10.0*f << endl;
//Umwandlung von i nach float, dann Division u. Multiplikation
//Ausgabe: -0.04
```

0.4.7.3 Beispiel: Explizite Typumwandlungen in C++

Besser ist explizite Konversion mittels `static_cast<>()`.

```
int i1 = 1;
int i2 = 2;

float f = i1 / i2;
cout << "f = " << f << endl;
  //int-Division, Ausgabe: 0

f = static_cast<float> (i1) / i2;
cout << "f = " << f << endl;
  //Umwandlung von i1 nach float, float-Division
  //Ausgabe: 0.5

int i3 = static_cast<int> (static_cast<float> (i1) / i2 + 0.5);
cout << "i3 = " << i3 << endl;
  //Addieren von 0.5 für mathematisch korrekte Rundung
  //Explizite Umwandlung des Ergebnisses nach int (sonst Warnung)
  //Ausgabe: 1
```

© R.-R. Grigat 2003-2019

0.4.7.4 Arten von expliziten Typumwandlungen in C++

Explizite Typumwandlung in C++:

- `static_cast<Zieltyp>(Operand)`
  - Typumwandlung zur Kompilierzeit
  - Umwandlung zwischen verwandten Typen z. B. **`float`** nach **`int`**

  Beispiel: KP24_1_productFood_UpDowncast_Aufgabe

- `dynamic_cast<Zieltyp>(Operand)`
  - Typumwandlung (nur) zwischen *Zeigern oder Referenzen* auf verwandte *polymorphe* Klassen
  - Typumwandlung zur Laufzeit
  - Ermöglicht herauszufinden, Objekt welcher abgeleiteten Klasse (z. B. `CCabrio*`, oder `CBus*`) in Variable vom Typ der Basisklasse (z. B. `CAuto*`) gespeichert ist.
  - Falls Umwandlung in Zeigertyp fehlschlägt, wird 0 zurückgeliefert
  - Falls Umwandlung in Objekt oder Referenz(C++) fehlschlägt, wird **`bad_cast`** Ausnahme geworfen

  Beispiel: KP25_3_dynamicCast

- `reinterpret_cast<Zieltyp>(Operand)`
  - Typumwandlung zwischen nicht verwandten Typen
  - *Beispiel:*
    - **`int`** nach Zeiger
    - Zeiger eines Typs in Zeiger eines anderen Typs
  - *Nicht portabel!*

- `const_cast<Zieltyp>(Operand)`
  - Entfernt **`const`**

© R.-R. Grigat 2003-2019

0.4.7.5 Konvertierungskonstruktor

Konvertierungskonstruktor

- Konstruktor mit Argumenten, deren Typ nicht identisch zum Typ der eigenen Klasse ist.
- Steht kein passender Konvertierungskonstruktor zur Verfügung, dann kombiniert Compiler mit impliziter Konvertierung, falls möglich.

```
1  class Euro{
2  Euro(int);     // Konvertierungskonstruktor int -> Euro
3  Euro(double); // Konvertierungskonstruktor double -> Euro
4  }
5  Euro anzahl(100);    // int -> Euro
6  Euro saldo(123.45); // double -> Euro
7  anzahl = 2.7;        // Implizite Konvertierung double -> Euro
8  saldo  += 20;        // Implizite Konvertierung int -> Euro
9  saldo = Euro(23.56); // Explizite Konvertierung (Konstruktorschreibweise)
10 saldo = (Euro)23.56; // Explizite Konvertierung (Cast-Schreibweise)
11 saldo = anzahl;      // Keine Konvertierung (Zuweisung)
```

0.4.7.6 Konvertierungsfunktion versus Konvertierungskonstruktor

Konvertierungsfunktion

- Konvertierungskonstruktor: anderer Typ -> aktuelle Klasse
- Konvertierungs*funktion*: aktuelle Klasse -> anderer Typ
- Name der Konvertierungsfunktion: `"operator Zieltyp"`

```
1  class Euro{
2     long data;  // in Cents
3  public:
4     Euro(int);  // in Cents
5     operator double() const { return (double)data/100.0; }
6  }
7  int main(){
8  Euro gehalt(54321);
9  double x = gehalt;     // implizit Euro -> double, 543.21
10 x = (double)gehalt;    // explizit Euro -> double, 543.21
```

© R.-R. Grigat 2003-2019

```
11 x =  double(gehalt);  // Konstruktorschreibweise, 543.21
12 x = gehalt.operator double();     // auch möglich, 543.21
13 int i = gehalt;        // Euro -> double -> int,    543
14 }
```

Beispiel: KP20_1_Konvertierungsfunktion

0.4.7.7 Mehrdeutigkeit bei Typumwandlungen

Mehrdeutigkeit und explicit

- Konvertierungsfunktion kann Mehrdeutigkeit bewirken
- Implizite Konvertierung nur bei Eindeutigkeit möglich
- Durch **explicit** wird ein Konvertierungskonstruktor oder eine Konvertierungsfunktion für implizite Verwendung gesperrt.
- Alternativ statt einer Konvertierungsfunktion eine entsprechend benannte Methode asDouble() oder ähnlich definieren, diese kann Compiler nicht implizit verwenden

```
1 explicit Euro(double x);  // Euro(double) nur noch explizit nutzbar
2 double asDouble()const {return (double)data/100.0;}   // nur explizit
3 explicit operator double()const {return (double)data/100.0;} // nur explizit
```

Beispiel: KP20_2_explicit_Konvertierungsfunktion

0.4.7.8 Beispiel explicit

```
1 struct A {          A(int) { }      // converting constructor
2                     A(int, int) { } // converting constructor (C++11)
3                     operator bool() const { return true; }   };//conv. fct.
4 struct B {  explicit B(int) { }
5             explicit B(int, int) { }
6             explicit operator bool() const { return true; }   };
7 int main() {
8    A a1 = 1;      // OK: copy-initialization selects A::A(int)
9    A a4 = {4, 5}; // OK: copy-list-initialization selects A::A(int, int)
10   if (a1) ;      // OK: A::operator bool()
11   bool na1 = a1; // OK: copy-initialization selects A::operator bool()
12   bool na2 = static_cast<bool>(a1); // OK: static_cast, direct-initialization
13 // B b1 = 1;      // error: copy-initialization, does not consider B::B(int)
```

© R.-R. Grigat 2003-2019

```
14    B b2(2);          // OK: direct-initialization selects B::B(int)
15    B b3 {4, 5};      // OK: direct-list-initialization selects B::B(int, int)
16 // B b4 = {4, 5}; // error: copy-list-initialization, not B::B(int,int)
17    B b5 = (B)1;      // OK: explicit cast performs static_cast
18    if (b2) ;         // OK: B::operator bool()
19 // bool nb1 = b2; // error: copy-initialization, not B::operator bool()
20    bool nb2 = static_cast<bool>(b2); // OK: static_cast, direct initial.
21 }
```

Beispiel: explicit_cppreference.com

0.5 Generische Programmierung

0.5.1 Einleitung

Definition: Generische Programmierung
Generische Programmierung ist Programmierung von Algorithmen mit noch unbekannten Datentypen

- Datentypen werden als Parameter angegeben
- Programmierung für mehrere Datentypen wird möglich
- *C++ Templates* bieten direkte Unterstützung für generische Programmierung

0.5.2 C++ Templates

Beispiel: Funktions-Template derGroessere

```
1 // Template-Parameter T stellt einen noch unbekannten Datentyp dar
2 template <typename T>
3 const T& derGroessere(const T& a, const T& b) {
4   return a > b ? a : b;
5 }
```

- `template<typename T>` deklariert ein (Funktions-)Template
- T ist Template-Parameter, ein noch unbekannter Datentyp
- *Template-Instantiierung:* Generieren einer Template-*Funktion* (Funktionsdefinition) durch den *Compiler* aus einem Funktions-*Template* mit Festlegung der Template-Parameter
- Entsprechend wird eine Klasse aus einem Klassen-Template und aus Template-Argumenten durch den Compiler generiert
- C++ Templates (*Vorlagen, Schablonen*) erlauben es, einen Typ als Parameter für eine Klasse oder für eine Funktion anzugeben
- Statt `<typename T>` darf auch `<class T>` geschrieben werden (nicht empfohlen)
  - `<typename T>` ist „sauberer", da T ein Typname ist
  - T muss kein Klassenname sein
  - Übergabe eines `int` wäre beispielsweise auch erlaubt

Beispiel: KP32_2_minimumFunktionstemplate

© R.-R. Grigat 2003-2019

0.5.2.1 Template-Klassen und Template-Parameter

- Template-Parameter können Datentypen (**typename** T) aber auch *Konstanten* repräsentieren (z. B. **int** n, aber kein Gleitpunkt-Typ zulässig)
- Datentypen und Konstanten auch kombiniert möglich
- Bei der Template-Instantiierung müssen Konstanten einen bekannten Wert besitzen
- Default-Argumente zulässig (Regeln wie bei Funktionen)

Beispiel

```
1 //noch unbekannt:
2 //T (Datentyp), n (Konstante)
3 template <typename T=double, int n=10>
4 class CPicture
5 {
6   /* ... */
7 };
```

```
1 int i=5;
2 CPicture<int, 256> Pic1;
3     // Okay, 256 ist Konstante
4     // und Typ int
5 CPicture<int, i  > Pic2;
6     // Fehler, i ist nicht const
```

0.5.2.2 Template-Argumente, die keine Typnamen sind

Template-Argumente (keine Typnamen)

- Als Referenz nur *globales* oder *statisches* Objekt zulässig
- Als Zeiger nur Adresse von *globalem* Objekt oder *globaler* Funktion zulässig
- Ansonsten nur *konstanter* Ausdruck zulässig
- Zeichenketten (d.h. **char***) wie "Oktoberfest" nicht zulässig, da nicht global, sondern statisch. Globaler String ist dagegen erlaubt.

```
1 char str[] = "Laterne";  // global
2 Demo<double, str> abc;   // ok
```

© R.-R. Grigat 2003-2019

0.5.2.3 Memberfunktions-Templates

- Memberfunktionen in Klassen-Templates sind per Definition *Funktions-Templates* und werden wie Funktions-Templates behandelt
- Definitionen der Memberfunktionen *müssen in der Header-Datei* erfolgen, wegen Risiko von Mehrfachdefinitionen und Linkerfehlern.
  - Compiler führt Ersetzung der Template-Parameter (nur) für alle tatsächlich benötigten Datentypen durch

© R.-R. Grigat 2003-2019

Beispiel

```
1  //block.hpp
2  template <typename T> class CBlock
3  {
4  public:
5     CBlock(int num);
6     /* ... */
7  private:
8           T* elem;
9           int n;
10 };
```

```
12 //Template-Konstruktor
13 template<typename T>
14       CBlock<T>::CBlock(int num=100):
15       n(num)
16 {
17    elem = new T[num];
18 }
19
20 /* ... */
```

0.5.2.4 Instantiierung versus Spezialisierung bei Klassen-Templates

Definition Template-*Instantiierung*:
Generieren der Klassendefinition aus Klassen-Template nach Einsetzen (*Spezifizierung*) ihrer Template-Parameter

Definition Template-*Spezialisierung*:
Eine abweichende Version eines Templates für spezielle individuelle *Template-Argumente* (die noch Template-Parameter enthalten können)

Beispiel

```
1 //Instantiierung
2 CBlock<int    > intBlk;
3 CBlock<double> doubleBlk;
4 template<> class CBlock<bool>{       // Komplette Spezialisierung
5 // template<> bedeutet, dass alle Template-Parameter vorab bekannt sind,
6 // d.h. nicht mehr spezifiziert werden müssen,
7 // <bool> spezifiziert den Template-Parameter in dieser Spezialisierung
```

© R.-R. Grigat 2003-2019

```
8 ...
9 };
```

0.5.2.5 Komplette versus partielle Spezialisierung

Template-Parameter T verbleibt nur bei partieller *Spezialisierung zu spezifizieren*

```
1 template<>          class CBlock<int>{ ... };  // komplette Spezialisierung
2 template<typename T> class CBlock<T*> { ... };  // partielle Spezialisierung
```

Beispiel: Einschränkung auf Zeigertypen

```
1 template<typename T> class CBlock<T*>{ .... }; // partielle Spezialisierung
2 // Template-Argument ist T*, Template-Parameter ist T
3 // Zur Instantiierung muss T noch spezifiziert werden.
4
5 // Instantiierungen des Klassen-Templates
6 // Template-Parameter T ist nicht identisch zu Template-Argument
7 CBlock<int* >  intBlk;  // <T*> ist <int*>,  damit ist T ein int
8 CBlock<int**> pintBlk;  // <T*> ist <int**>, damit ist T ein int*
```

Beispiel: KP32_2_minimumSpezialisierung

0.5.2.6 Template-Funktionen und Template-Klassen

- Bei der Template-Instantiierung von Klassen-Templates sind alle Template-Parameter *explizit* anzugeben
- Funktions-Templates sollten Typ der Template-Parameter *implizit* aus den Funktionsargumenten ermitteln können (sonst sind Template-Parameter explizit anzugeben!)

Beispiel

```
1 template<typename T, int n> void sort(CPicture<T, n>& Pic)
2 { /* ... */ }
3
4 int main()
5 { //Klassen-Template: Template-Parameter muessen angegeben werden
6   CPicture<int, 256> intPic;
7   CPicture<char, 26> charPic;
8 ...
```

© R.-R. Grigat 2003-2019

```
9   //Funktions-Template: Template-Parameter sind wegen der
10  //Funktions-Argumente hier implizit eindeutig
11  sort(intPic);  // Template-Funktion aus Funktions-Template
12  sort(charPic); // Andere Template-Funktion, selbes Template
13 ...
14 }
```

0.5.2.7 Explizite Spezifizierung von Template-Funktionen

- Explizite Angabe aller Template-Parameter hinter Funktionsname zwingend, falls nicht implizit zu ermitteln (nur falls auch tatsächlich in Funktion verwendet!).
- In Liste der expliziten Template-Parameter dürfen Argumente (nur von hinten) weggelassen werden.
- Häufige Anwendung ist Angabe des Rückgabetyps von Template-Funktion.

Beispiel (nach Stroustrup)

```
1  // Funktions-Template implicit_cast
2  template<typename T, typename U> T implicit_cast(U u){return u;}
3  void g(int i) {
4    // Explizit spezifizierte Template-Funktionen implicit_cast
5    implicit_cast(i);              // Fehler: kann T nicht ermitteln
6    implicit_cast<double>(i);      // T ist double, U ist int
7    implicit_cast<char,double>(i); // T ist char, U ist double
8    implicit_cast<char*,int>(i);   // T ist char*; U ist int
9                                   // Fehler: kann nicht int nach char* wandeln
10 }
```

Beispiel: KP32_3_quadraticMatrixDefaultArguments

0.5.2.8 Überladen von Funktions-Templates

Überladen von Funktions-Templates und normalen Funktionen

- Funktions-Templates mit demselben Namen, aber unterschiedlicher Signatur
- Funktions-Templates und normale Funktionen mit demselben Namen

© R.-R. Grigat 2003-2019

```
1 template<typename T> T max(T, T);  // Funktions-Template
2 double max(double, double);        // "normale" Funktion
3
4 max(3, 5);          // max<int>(int, int)
5 max('c', 'z');      // max<char>(char, char)
6 max(6.2, 3.7);      // max(double, double), "normale" Funktion vor Template
7 max(1, 'b')         // Fehler, mehrdeutig, aber:
8 max<int>(1, 'b'); // max<int>(1, int('b'))
```

0.5.2.9 Überladen von Funktions-Templates (2)

Regeln beim Auflösen von Mehrdeutigkeiten

- Ermittle Menge aller zur Auflösung geeigneten *Spezifizierungen* von Funktions-Templates
- Reduziere Lösungsmenge auf die Auflösungen mit *höchstem Grad* der Template-*Spezialisierung*, also Auflösung mit stärkster Einschränkung.
- Löse Überladungen aus obiger Lösungsmenge und aus Lösungsmenge normaler Funktionen nach den *Regeln für normale Funktionen* auf. Ohne Typkonversion (cast) passende Lösungen werden bevorzugt.
- Falls eine normale Funktion und eine Template-Spezialisierung gleich gute Übereinstimmungen liefern, so wird die *normale Funktion bevorzugt*.
- Fehler, falls keine oder mehrere gleichrangige beste Übereinstimmungen existieren.

0.5.2.10 Variadische Templates

Parameterpacks

- *Variadische Templates* besitzen variable Parameterzahl
- *Template-Parameterpack* enthält Typen der variablen Template-Parameter `template<typename ...T`
- *Funktions-Parameterpack* enthält die zugehörigen Funktionsparameter `void g(Types ...args){}`
- `g()` hat $n \geq 0$ Argumente unterschiedlichen Typs

© R.-R. Grigat 2003-2019

- Nachgestellte Punkte übersetzt Compiler in die Liste der Aufrufparameter, `g(args...)` ergibt `g(args1, args2, args3)`

```
template<typename ...Types>
void g(Types ...args){ /* Funktionsblock */ }
```

0.5.2.11 Rekursives Entpacken

Rekursives Entpacken variadischer Templates

Rekursives Entpacken einer Funktion erfordert

- *Basisfall* ohne Parameterpack und
- *allgemeinen Fall* mit Parameterpack.

Im *allgemeinen Fall* ruft die Funktion sich rekursiv auf mit jeweils verringerter Parameterzahl bis Basisfall erreicht ist.

```
template<typename T>                          // Basisfall
T add(const T& a){return a;}
template<typename T, typename ... Args>  // Rekursion
T add(const T& first, const Args& ...rest)
{return first + add(rest...);}
int main(){
    cout << add(1, 2, 3, 4) << endl;  // 10
    string s1 ="Sonnen", s2 ="blumen", s3 ="kerne";
    cout << add(s1, s2, s3) << endl;  // Sonnenblumenkerne
    return 0; }
```

Beispiel: KP32_4_variadicTemplates

0.5.2.12 Klassentemplates als Basisklasse

Klassentemplates als Basisklasse

- Basisklasse `vector<T>` ist Templateklasse, daher:
  - `begin()`, `end()` etc. voll qualifiziert aufrufen `vector<T>::begin()` oder `SortVec::begin()`
  - Funktionen der Basisklasse benötigen **typename** (weil im Beispiel `iterator` kein Wert ist, sondern Typ)

© R.-R. Grigat 2003-2019

```
template <typename T, typename Compare = less<T>>
class SortVec : public vector<T> {  // Basisklasse ist Templateklasse
  public:
    SortVec(int n, const T& x = T()) : vector<T>(n,x) {}
    void insert(const T& obj);          // Sortiert Einfügen
};
template <typename T, typename Compare >
void SortVec<T, Compare>::insert(const T& obj) {
    typename SortVec::iterator  pos, temp;  // typename erforderlich
    SortVec::push_back(obj);                // SortVec:: erforderlich
    pos = SortVec::end()-1;                 // SortVec:: erforderlich
...
```

Beispiel: KP33_1_binarySortMergeVector

0.5.2.13 Überladene Funktion als Aufrufparameter

Überladene Funktion als Aufrufparameter

- Funktion `sqrt()` aus `cmath` ist für **double** *überladen*, die Template-Algorithmen benötigen aber Funktionen mit *expliziter* Festlegung des Typs, unten z.B. **double**.
- Daher muss ein cast auf korrekten Aufruftyp und Rückgabetyp erfolgen.

```
static inline double computeSqrt (double x) { return sqrt(x); }
transform(dblVec1.begin(), dblVec1.end(), dblVec2.begin(),
          computeSqrt);  // ok, double(double)

transform(dblVec1.begin(), dblVec1.end(), dblVec2.begin(),
          sqrt);  // Fehler, unresolved overloaded function type
transform(dblVec1.begin(), dblVec1.end(), dblVec2.begin(),
          (double(*)(double)) sqrt);  // ok, cast auf double(*)(double)
transform(dblVec1.begin(), dblVec1.end(), dblVec2.begin(),
          ptr_fun<double, double>(sqrt));
                                // ok, cast mittels std::ptr_fun<T,T>()
```

Beispiel: KP_36_08_transform

© R.-R. Grigat 2003-2019

0.5.3 C++-Standardbibliothek

C++-Standardbibliothek std

- Namespace `std` sowie Satz von Headern
- Die grundlegende Programmbibliothek von C++
- Bestandteil des ISO C++-Standards
- Historisch: 1998 Aufnahme der HP Standard Template Library (STL) in die Standardbibliothek des C++-Standards
- Effiziente Algorithmen und Datenstrukturen

Literatur

- `https://de.wikipedia.org/wiki/C++-Standardbibliothek`
- `http://en.cppreference.com/w/`
- Nicolai M. Josuttis.
  „The C++ Standard Library - A Tutorial and Reference“

0.5.3.1 Komponenten der Standardbibliothek

Drei Haupt-Komponenten

- Container.
  - Behälter für Daten, z. B. Stack, Liste usw.
  - Template-Klassen
- Iteratoren
  - ähneln Zeigern (Operatoren *, ++, ==, !=, =, z. T. --, [])
  - bewegen sich schrittweise durch die Elemente von Containern
  - erlauben Zugriff auf das Element, auf das sie zeigen
  - sind implementiert als (Sub-)Klasse innerhalb jeder Containerklasse
  - bilden ein gemeinsames Interface für beliebige Containertypen
- Algorithmen

© R.-R. Grigat 2003-2019

- suchen, sortieren, manipulieren Daten
- greifen auf Daten mittels Iteratoren zu
- *global* definierte Funktionen mit Iteratoren als Parameter
- verletzen also objektorientiertes Programmiermodell

0.5.3.2 Trennung von Datenstrukturen und Operationen

- Daten werden von Containern verwaltet
- Operationen werden durch Algorithmen definiert
- Iteratoren sind die Verbindung zwischen Containern und Algorithmen

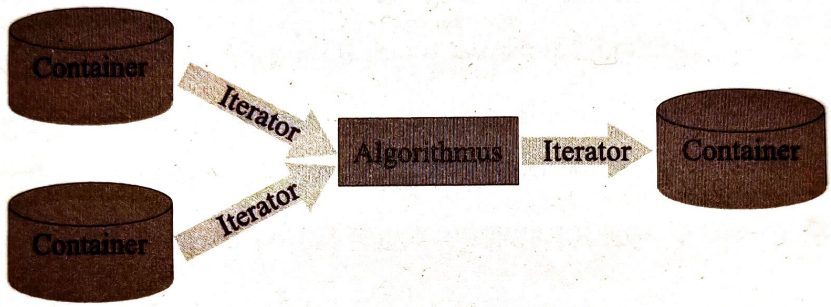


Abbildung 12: Komponenten der Standardbibliothek

0.5.4 Smart Pointers

0.5.4.1 Smart Pointer

Smart Pointer

- Intelligente Zeiger auf (meist) dynamische Objekte
- Vermeidung von Speicherlecks
- Objekt zerstören, wenn letzter Zeiger darauf zerstört wird
- keine Zeigerarithmetik mit Smart Pointern
- Header <memory>
- Drei Zeigerklassen der Standardbibliothek

© R.-R. Grigat 2003-2019

- `unique_ptr<typename T, typename D=default_delete<T>>` nicht kopierbar, alleiniger Besitzer
- `shared_ptr<typename T>` kopierbar, einer der Besitzer
- `weak_ptr<typename T>` kopierbar, ist nicht Besitzer

0.5.4.2 `unique_ptr<class T, class D = default_delete<T>>`

`unique_ptr<typename T, typename D=default_delete<T>>`

- Verwaltung von Ressourcen (Speicher, Dateien, Geräte)
- Template-Parameter D ist Funktion (oder Funktor) zur Ressourcenfreigabe mit Defaultwert `default_delete<T>`
- `default_delete<T>` gibt mit **`delete`** *Speicher* frei
- `T* get();` gespeicherter Zeiger
- `D& get_deleter();` Referenz auf Deleter-Funktion
- **`void`** `swap(unique_ptr& up);` Besitzertausch
- `T* release();` gibt Besitz ab (liefert alten Zeigerwert, setzt Zeiger auf Null)
- **`void`** `reset(T* p=nullptr);` nimmt Objekt in Besitz (zerstört bisheriges Objekt, übernimmt Zeigerwert p)
- `up1.reset(up2.release());` ist *Move-Zuweisung*, äquivalent zu `up1=std::move(up2);`
- Call by Value und Rückgabe nur mit *Move-Konstruktor*

0.5.4.3 `unique_ptr`

Move-Konstruktor und Move-Zuweisung

- `std::move()` erzeugt R-Wert (hier X-Wert) aus L-Wert
- Move-Konstruktor: `unique_ptr<int> up2(std::move(up1))`
- Anschließend besitzt up2 das Objekt, up1==nullptr
- Kopierkonstruktor und Zuweisung verboten

© R.-R. Grigat 2003-2019

- keine Zeigerarithmetik

```
#include <memory>
unique_ptr<int> up1(new int(20)), up2; // *up1==20, up2==nullptr
// unique_ptr<int> up2(up1);          // Fehler, kein Kopierkonstruktor
unique_ptr<int> up2(std::move(up1)) // ok, Move-Konstruktor, up1==nullptr
// up2++;                             // Fehler, keine Zeigerarithmetik
*up2 += 10;                           // Dereferenzierung ok, Objekt geändert
// up1 = up2;                         // Fehler, keine Zuweisung
up1 = std::move(up2);                 // ok, Move-Zuweisung, up2==nullptr
```

Beispiel: KP34_1_uniquePtr1, KP34_2_uniquePtr2, KP34_3_uniquePtr3

0.5.4.4 shared_ptr

Charakteristika von shared_ptr<T>

- Mehrere Smart Pointer vom Typ `shared_ptr<T>` können sich den Besitz an einem Objekt teilen
- Klasse `shared_ptr<T>` besitzt Referenzzähler für Zahl der Besitzer
  - `long use_count() const;` liefert Wert des Referenzzählers
  - `bool unique() const;` liefert wahr, wenn Pointer einziger Besitzer ist
- Klasse besitzt Kopierkonstruktor und Zuweisungsoperator
- Objekt wird zerstört, wenn letzter Smart Pointer Besitz abgibt

0.5.4.5 Konstruktoren der shared_ptr

Konstruktoren von shared_ptr<T>

- `shared_ptr<T> sp(new U);` möglich, wenn Typ `U*` in Typ `T*` konvertierbar ist
- Kontrollblock enthält den wahren Typ, so dass richtiger Destruktor aufgerufen wird
- Destruktor braucht nicht virtuell zu sein

```
shared_ptr<Base> pBase(new Derived);
pBase.reset();    // Derived-Destruktor!
```

© R.-R. Grigat 2003-2019

0.5.4.6 Operatoren der shared_ptr

Operatoren von shared_ptr<T>

- Zugriffsoperatoren * und -> existieren wie bei `unique_ptr`
- Zuweisung und Move-Zuweisung zulässig
- Vergleich von Smart Pointern ist Vergleich der Zeiger
- Implizite Konvertierung von `shared_ptr<U>` in `shared_ptr<T>`, wenn z.B. T Basisklasse von U ist.

```
1 shared_ptr<int> sp1, sp2(new int(100));
2 sp1=sp2;    // Zuweisung ist ok
3 cout << sp1;    // Ausgabe von sp1.get()
```

0.5.4.7 Methoden von shared_ptr

Methoden von shared_ptr<T>

- `release()` *nicht* vorhanden
- `use_count()`, `unique()`, `get()`, `reset()`, `swap()`, `get_deleter<D>()` vorhanden
- `get_deleter<D>()` liefert Zeiger auf Deleter (nur) falls vorhanden
- `make_shared<>()` Effiziente Factory Funktion für Objekt *und* Kontrollblock

```
1 shared_ptr<int> sp1, sp2(new int(10));
2 sp1.swap(sp2);    // *sp1==10 und sp2 leer
3 sp1.reset(new int(20)); // Gibt int(10) frei
4 sp1 = sp2;  //sp1.reset(); da sp2 leer
```

Beispiel: KP34_4_sharedPtr1, KP34_5_sharedPtr2, KP34_6_sharedPtr3

© R.-R. Grigat 2003-2019

0.5.4.8 Smart Pointer vom Typ `weak_ptr`

Smart Pointer vom Typ `weak_ptr<T>`

- Anwendungsbeispiel Auflösen zyklischer Verweise (Referenzzähler nicht 0, kein Zugriff mehr möglich)
- Kann nur auf Objekt in Besitz von `shared_ptr` verweisen
- Erzeugung mit `weak_ptr<int> wp(sp);`, Referenzzähler wird nicht verändert, Lebensdauer nicht beeinflusst
- `weak_ptr<T>`-Methode `expired()` zeigt an, ob Objekt bereits zerstört wurde
- Zugriff auf Objekt nur über `shared_ptr<T>`
  - `weak_ptr<T>`-Methode `lock()` gibt `shared_ptr<T>`-Objekt zurück (oder leeres `shared_ptr<T>`-Objekt und Konstruktor wirft Exception `bad_weak_ptr`)
  - Konstruktor von `shared_ptr<T>` akzeptiert Argument vom Typ `weak_ptr<U>` wenn `U*` in `T*` konvertierbar ist

Beispiel: KP34_7_weakPtr1

0.5.5 Komplexitäten und O-Notation

0.5.5.1 Komplexität von Algorithmen

Definition Komplexität:

- Komplexität eines Algorithmus ist der maximale Bedarf an
  - Rechenzeit
  - Speicherplatz
- wird als Funktion der Größe n der Eingabedatenmenge angegeben
- in dieser Vorlesung nur Betrachtung der *Rechenzeitkomplexitäten* $T(n)$
  - generell ist Zeitbedarf kritisch
  - Speicherbedarf meist nur kritisch bei Hardware-Implementierungen

© R.-R. Grigat 2003-2019

0.5.5.2 Analyse der Rechenzeit von Algorithmen

Analyse der Zeitbedarfs $T(n)$[2]

- Rechenzeitbedarf i. A. abhängig von Größe n der Eingabemenge
- Betrachtung der Spezialfälle
  - bester Fall *(best case)* T_{best}
  - schlimmster Fall *(worst case)* T_{worst}
- Betrachtung des Durchschnittsbedarfs *(average case)* T_{avg}
  - Häufigkeit oder Wahrscheinlichkeit der Eingaben berücksichtigen
  - Gewichtetes Mittel bilden

0.5.5.3 Beispiel zu Rechenzeitbedarf

Beispiel:

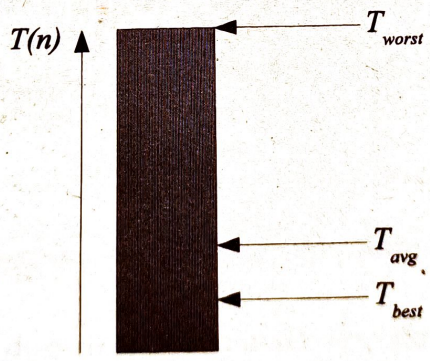


Abbildung 13: Rechenzeitbedarf für einen Algorithmus

- In diesem Beispiel tritt T_{worst} nicht so häufig auf
- T_{avg} näher an T_{best}

[2]Nicht verwechseln mit Templateparameter T

© R.-R. Grigat 2003-2019

0.5.5.4 *O*-Notation

Rechenzeitbedarf von Algorithmen

- Problemgröße n, Rechenzeiten $f(n)$, $g(n)$
- Rechenzeitbedarf $f(n)$ abschätzen durch Funktion $g(n)$
- Dazu additive und multiplikative Konstanten weglassen

Definition *O*-Notation:
Seien $f : \mathbb{N} \to \mathbb{R}^+$, $g : \mathbb{N} \to \mathbb{R}^+$

$$f \in O(g) : \Leftrightarrow \quad \exists\, n_0,\, c \in \mathbb{R},\, c > 0 : \quad \forall\, n \geq n_0 \quad f(n) \leq c \cdot g(n)$$

- „f wächst höchstens so schnell wie g“
- Unpräzise Schreibweise: $f = O(g)$ „f ist $O(g)$“
- Präzise Schreibweise: $f \in O(g)$

$$O(g) = \{f : \mathbb{N} \to \mathbb{R}^+ \mid \exists\, n_0 \in \mathbb{N},\, c \in \mathbb{R}^+ : \forall\, n \geq n_0 \quad f(n) \leq c \cdot g(n)\}$$

0.5.5.5 Bestimmung von Komplexitäten

Beispiel: $f(n) \equiv T_1(n)$, *gesucht* $g(n), n_0, c$

$$T_1(n) = n + 5 \quad \in O(n), \quad \text{also} \quad g(n) = n$$

$$n_0 + 5 \leq c \cdot n_0 \Rightarrow n_0 \geq \frac{5}{c-1} \quad \forall\, c > 1$$

Komplexität mittels Grenzwert $\lim\limits_{n\to\infty} \frac{f(n)}{g(n)}$ *bestimmen*

- Für die Komplexitäten gilt $f(n) > 0$ und $g(n) > 0$
- Grenzwert existiert $\Longleftrightarrow$ $\lim\limits_{n\to\infty} \frac{f(n)}{g(n)} < \infty \Longleftrightarrow f \in O(g)$

$$f(n) \equiv T_2(n) = 2n^2 + 100n + 10000 \quad \in O(n^2), \quad \text{also} \quad g(n) = n^2$$

$$\lim_{n\to\infty} \frac{2n^2 + 100n + 10000}{n^2} = 2$$

© R.-R. Grigat 2003-2019

0.5.5.6 Beispiel zu *O*-Notation

Beispiel:

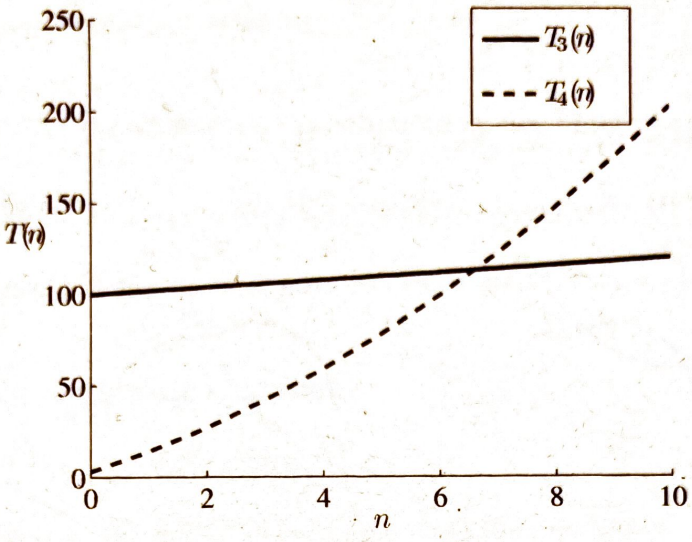


Abbildung 14: Beispiel zu O-Notation

$$T_3(n) = 2 \cdot n + 100 \qquad \in O(n)$$
$$T_4(n) = 2n^2 + 10 \cdot n + 4 \qquad \in O(n^2)$$

- Für kleine n Terme niedriger Ordnung entscheidend!
- Hier: Für $n \leq 6 : T_4(n) < T_3(n)$

0.5.5.7 Eigenschaften der *O*-Notation

***O*-Notation**

- **Elimination von Konstanten**
  - $O(n) = O(\frac{n}{2}) = O(2n) = O(3n+5)$
  - Man schreibt $O(n)$ für alle diese Mengen
- **Bilden von oberen Schranken**
  - $O(1) \subset O(\log n) \subset O(n) \subset O(n^2) \subset O(2^n)$

© R.-R. Grigat 2003-2019

- Hinweise zur *mathematisch falschen* Schreibweise:
  - „Gleichung“ $f = O(g)$ nur *von links nach rechts* lesen
  - Aussage $O(g) = f$ ist *sinnlos*!
  - $O(1) = O(n) = O(n^2) = O(2^n)$ nur von links nach rechts lesen

0.5.5.8 Rechenregeln der O-Notation

Addition von Funktionen

$$O(g_1(n)) \cup O(g_2(n)) = O(g_1(n) + g_2(n))$$
$$= \begin{cases} O(g_1(n)) & \text{falls } g_2 \in O(g_1) \\ O(g_2(n)) & \text{falls } g_1 \in O(g_2) \end{cases}$$

0.5.5.9 Einteilung von Laufzeitfunktionen in Komplexitätsklassen

Sprechweise	Klasse	Beispiel
konstant	$O(1)$	
logarithmisch	$O(\log(n))$	Binäre Suche
linear	$O(n)$	
log-linear	$O(n \cdot \log(n))$	Schnelle Sortierverfahren
quadratisch	$O(n^2)$	Primitive Sortierverfahren
polynomiell	$O(n^k)$, $k \geq 2$	
exponentiell	$O(2^n)$	

Tabelle 5: Komplexitätsklassen

© R.-R. Grigat 2003-2019

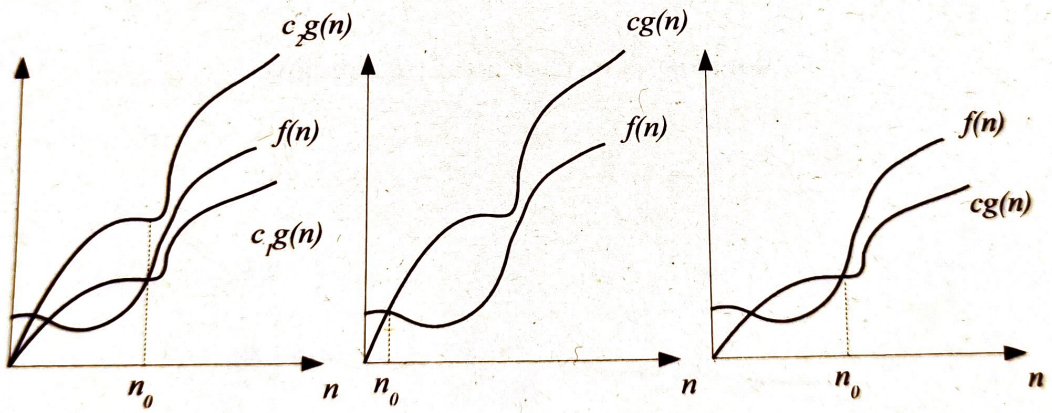
0.5.5.10 Allgemeine *O*-Notation

Definition:

präzise	falls	„kurz“	„Merkregel“
$f \in o(g)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$	„$f < g$“	„f wächst langsamer als g“
$f \in O(g)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$	„$f \leq g$“	„f wächst höchstens so schnell wie g“
$f \in \Theta(g)$	$f \in O(g)$, $g \in O(f)$	„$f = g$“	„f und g wachsen gleich schnell (größenordnungsmäßig)“
$f \in \Omega(g)$	$g \in O(f)$	„$f \geq g$“	„f wächst mindestens so schnell wie g“
$f \in \omega(g)$	$g \in o(f)$	„$f > g$“	„f wächst schneller als g“

Tabelle 6: Allgemeine *O*-Notation

0.5.5.11 Beispiel zu allgemeiner *O*-Notation



(a) $f(n) \in \Theta(g(n))$ (b) $f(n) \in O(g(n))$ (c) $f(n) \in \Omega(g(n))$

Abbildung 15: Graphisches Beispiel zu Θ-, *O*-, und Ω-Notation

© R.-R. Grigat 2003-2019

0.5.6 Container

0.5.6.1 Containerklassen

- Sequentielle Container

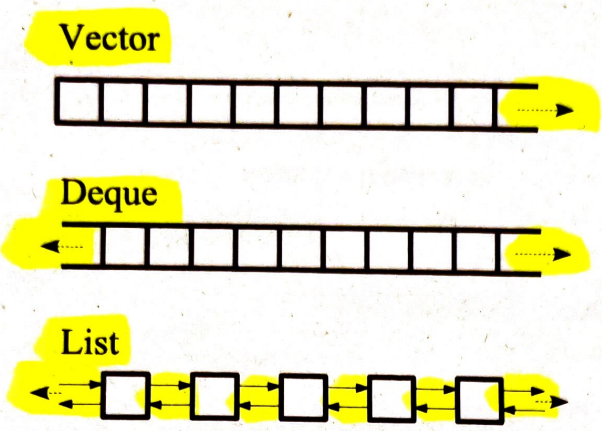


Abbildung 16: Sequentielle Container

– geordnete (nicht notwendig sortierte) Menge von Elementen
– die Position eines Elements ist unabhängig seinem Wert

- Assoziative Container

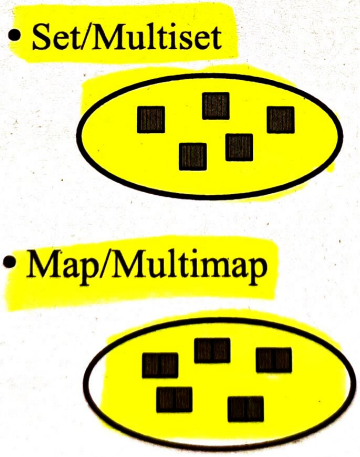


Abbildung 17: Assoziative Container

– sortierte Menge von Elementen
– die Position eines Elements hängt ab von seinem Wert

© R.-R. Grigat 2003-2019

0.5.6.2 Referenzen

Referenzen

- `http://en.cppreference.com/w/cpp/container`
- `http://www.cplusplus.com/reference/`
- `https://en.wikipedia.org/wiki/Sequence_container_(C%2B%2B)`
- `https://en.wikipedia.org/wiki/Associative_containers`
- `https://en.wikipedia.org/wiki/Unordered_associative_containers_(C%2B%2B)`

Welchen Container sollte man benutzen?

- *Vector*: Wahlfreier Zugriff, Einfügen am Ende
- *Deque*: Wahlfreier Zugriff, Einfügen an Anfang oder Ende
- *List*: Einfügen auch *in der Mitte*
- *Set/Multiset* oder *Map/Multimap*: Suche nach Inhalten
- Container *nicht* als Basisklasse für Ableitungen optimiert

0.5.6.3 Sequentielle Container – Vektoren

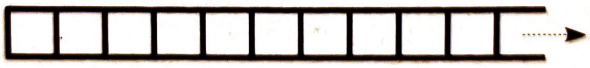


Abbildung 18: Struktur eines Vektors

- `vector`: dynamisches Feld
- Direkter Zugriff per `[]` ist möglich
- Elemente einfügen und entfernen
  - am Ende: $O(1)$
  - am Anfang: $O(n)$
  - in der Mitte: $O(n)$

© R.-R. Grigat 2003-2019

```
#include <vector>

int main()
{
  std::vector<int> vec;
  for(int i = 1; i <= 5; ++i)
  {
    vec.push_back(i);
  }
  std::cout << vec[0];//Ausgabe: 1
}
```

0.5.6.4 Sequentielle Container – Deque

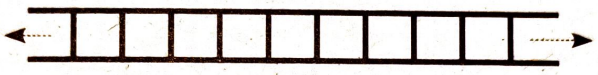


Abbildung 19: Struktur einer Deque

- deque: dynamisches Feld
- Direkter Zugriff per [ ] ist möglich
- Elemente einfügen und entfernen
  - am Anfang: $O(1)$
  - am Ende: $O(1)$
  - in der Mitte: $O(n)$

```
#include <deque>

int main()
{
  std::deque<float> deq;
  for(int i = 1; i <= 5; ++i)
  {
    deq.push_front(i*1.1);
    deq.push_back(i*2.1);
  }
  std::cout << deq[0];//Ausgabe: 5.5
}
```

© R.-R. Grigat 2003-2019

0.5.6.5 Sequentielle Container – Listen

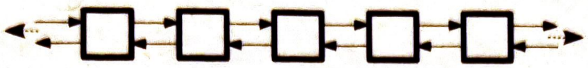


Abbildung 20: Struktur einer Liste

- `list`: Doppelt verkettete Liste
- Direkter Zugriff per `[]` ist *nicht* möglich
- Elemente einfügen und entfernen
  - am Anfang: $O(1)$
  - am Ende: $O(1)$
  - aktuelle Position: $O(1)$
  - andere Position: $O(n)$

```
1  #include <list>
2
3  int main()
4  {
5    std::list<char> lst;
6    for(char c = 'a'; c <= 'z'; ++c)
7    {
8      lst.push_front(c);
9    }
10   while(!lst.empty())
11   {
12     lst.pop_front();
13   }
14 }
```

0.5.6.6 Speichermodell

Allokator-Klasse

- Speichermodell wird durch Allokator-Klasse dargestellt
- Dynamische Speicherverwaltung für Objekte vom Typ T
- Default ist Standard-Allokator-Klasse `allocator<T>`

© R.-R. Grigat 2003-2019

- reserviert Speicher mit **new**
- gibt Speicher frei mit **delete**

Beispiele

```
vector<typename T, typename Allocator = allocator<T> >
list<typename T, typename Allocator = allocator<T> >
deque<typename T, typename Allocator = allocator<T> >
```

0.5.6.7 Assoziative Container – Set/Multiset

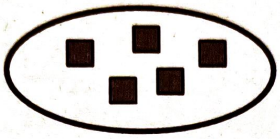


Abbildung 21: Die Struktur von Set/Multiset

- Elemente werden nach ihren Werten sortiert
- Für schnelles Suchen optimiert: $O(\log(n))$
- Duplikate :
  - Set: verboten
  - Multiset: erlaubt

```
#include <set>

int main()
{
  std::multiset<int> intSet;
  for(int i = 1; i <=5; i ++)
  {
    intSet.insert(i);
    intSet.insert(i*2);
  }
}
```

Beispiel: KP33_4_setMultiset

© R.-R. Grigat 2003-2019

0.5.6.8 Assoziative Container – Map/Multimap

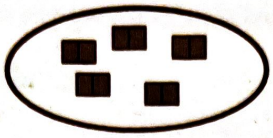


Abbildung 22: Die Struktur von Map/Multimap

- Elemente
  - vom Typ `pair<T1, T2>`
  - T1 ist Sortierschlüssel, T2 ist zugehöriges Element
  - Elemente T2 sind nach Schlüsseln sortiert abgelegt
- Für schnelles Suchen optimiert: $O(\log(n))$
- Duplikate:
  - Map: verboten
  - Multimap: erlaubt

Beispiel: KP33_5_mapMultimap, KP33_6_initializerList_RangeFor

Beispiel

```
#include <map>
#include <string>

int main()
{
  std::map<int, std::string> Student;
  Student.insert(std::make_pair(18789, "Daniel"));
  Student.insert(std::make_pair(16945, "Anna"));
  Student.insert(std::make_pair(20852, "Patrik"));
  Student.insert(std::make_pair(21953, "Michael"));
}
```

- `make_pair` ist global definierte Template-Funktion, die ein Objekt vom Typ `pair<T1, T2>` zurückliefert

© R.-R. Grigat 2003-2019

0.5.6.9 Ungeordnete assoziative Container

Unordered Set/Multiset/Map/Multimap

- Unsortiert gespeichert mittels Hashfunktion
- O(1) solange Speicherfüllungsgrad gering
- Individuelle Klassen erfordern eigene Hashfunktion
- Ansonsten identisch zu Set/Multiset und Map/Multimap

```
#include <iostream>
#include <string>
#include <unordered_map>
int main() {
    std::unordered_map<std::string, int> months;
    months["january"] = 31;
    months["february"] = 28;
//...
    months["december"] = 31;
    std::cout << "september -> " << months["september"] << std::endl;
    return 0;
}
```

Beispiel: KP33_7_unordered_map, KP33_8_unorderedSet_Hashing

0.5.6.10 Anmerkung zum direkten Elementzugriff bei Containern

Zugriff mit []

- Existenz des Elements wird *nicht* überprüft

Zugriff mit at()

- at() ist Memberfunktion von vector, deque
- Falls Element nicht existiert, wird out_of_range Ausnahme geworfen
- Langsamer als [], aber sicherer!

Beispiel

```
std::vector<int> vec;
vec.push_back(0);
std::cout << vec[1]; //undefiniert, evtl. Ausgabe eines zufaelligen Wertes
std::cout << vec.at(1); // wirft out_of_range - Ausnahme
```

© R.-R. Grigat 2003-2019

0.5.7 Iteratoren

0.5.7.1 Definition: Iteratoren

Iterator

- Ein Iterator ist eine Abstraktion eines Zeigers auf ein Element einer Sequenz (Container)
- Iteratoren der Standardbibliothek (std) sind Klassen im Namespace von Containern zu deren Traversierung
- In diesem Kapitel werden nur std-Iteratoren behandelt und im folgenden als Iterator bezeichnet
- Jede Instanz eines std-Iterators repräsentiert eine Position in einem Container
- Jeder Container "kenntßwei Iteratorklassen:
  1. `container::iterator`
     traversiert durch die Elemente im Lesen/Schreiben-Modus
  2. `container::const_iterator`
     traversiert durch die Elemente nur im Lesen-Modus

0.5.7.2 Operatoren für Iteratoren

Die folgenden Operatoren definieren das Verhalten von Iteratoren (z. B. für die Iteratoren `pos1`, `pos2`)

- Inhaltsoperator `*`, z. B. `*pos1`
  - Zugriff auf Element an aktueller Position
- Inkrement und Dekrement `++pos1` bzw. `--pos1`
  - inkrementieren bzw. dekrementieren (schrittweise weiterlaufen)
- Vergleichsoperatoren `pos1==pos2` bzw. `pos1!=pos2`
  - Vergleich der durch zwei Iteratoren repräsentierten Positionen
- Zuweisungsoperator `pos1=pos2`
  - Zuweisung der durch Iterator `pos2` repräsentierten Position

Obige Operatoren für Iteratoren entsprechen zunächst denen für Zeiger in C und C++. Iteratoren sind aber „intelligente“ Zeiger und können über komplizierte Datenstrukturen iterieren.

© R.-R. Grigat 2003-2019

0.5.7.3 Container und Iteratoren

- Die Memberfunktion `begin()` eines Containers liefert *Iterator*, der auf erstes Element zeigt
- Die Memberfunktion `end()` eines Containers liefert *Iterator*, der auf Position *hinter* letztem Element zeigt
- `begin()` und `end()` sind Member aller Container

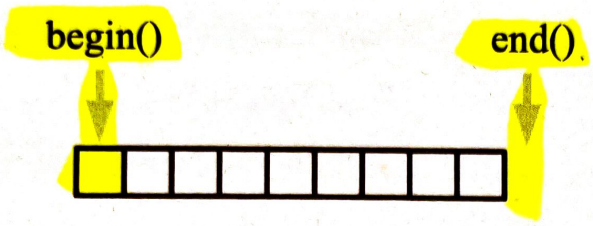


Abbildung 23: begin() und end() eines Containers

Iterator eines Containers vec vom Typ `vector<int>`

```
vector<int>::const_iterator pos;
for(pos = vec.begin(); pos != vec.end(); ++pos){
  cout<< *pos << ' ';
}
cout << endl;
```

Beispiel Iteratoren und Map

```
std::map<int, std::string> Student;
Student.insert(std::make_pair(18789, "Daniel"));
Student.insert(std::make_pair(16945, "Anna"));
Student.insert(std::make_pair(20852, "Patrik"));
Student.insert(std::make_pair(21953, "Michael"));

std::map<int, std::string>::iterator pos;
for(pos = Student.begin(); pos != Student.end(); ++pos)
{
  std::cout << pos->first << ':';
  std::cout << pos->second << std::endl;
}
std::cout << std::endl;
```

- `pos->first` wird vom Compiler umgesetzt zu (*pos).first (wie bei Zeigern)
- `first, second` sind die Membervariablen von `pair<T1, T2>`

© R.-R. Grigat 2003-2019

0.5.8 Traits

0.5.8.1 Traits der Standardbibliothek

Traits der Standardbibliothek

- Traits (Merkmale, Eigenschaften)
  - dienen dazu, Eigenschaften mit einem Typ zu verbinden
  - sind eine Art Bündel von Typfunktionen
- Standardbibliothek setzt intensiv Traits ein, wie `allocator_traits`, `char_traits`, `iterator_traits`, `regex_traits`, `pointer_traits`, `time_traits`, `type_traits`.

0.5.8.2 `iterator_traits`

- Template-Klasse `iterator_traits` definiert die Eigenschaften von Iteratoren

```
template<typename Iterator> struct iterator_traits {
    using difference_type = typename Iterator::difference_type;
    using value_type = typename Iterator::value_type;
    using pointer = typename Iterator::pointer;
    using reference = typename Iterator::reference;
    using iterator_category = typename Iterator::iterator_category;
};
```

- Abhängige Variablen deutet Compiler als Wert. `Iterator::difference_type` usw. sind aber *Typen*, daher direkt davor Schlüsselwort **typename** erforderlich.

```
template<typename Iter>
Iter search(Iter p, Iter q,
            typename iterator_traits<Iter>::value_type val) {
    typename iterator_traits<Iter>::difference_type m = q-p;
    // ...
}
```

0.5.8.3 Iterator mit einfacher Typfunktion

- Dasselbe Beispiel schlanker ohne **struct**.

© R.-R. Grigat 2003-2019

```
template<typename T>
using Value_type = typename std::iterator_traits<T>::value_type;
template<typename T>
using Diff_type = typename std::iterator_traits<T>::difference_type;
template<typename T>
using Iter_category= typename std::iterator_traits<T>::iterator_category;
template<typename Iter>
Iter search(Iter p, Iter q, Value_type<Iter> val) {
    Diff_type<Iter> m = q-p;
    ... }
```

- Noch schlanker mit Typfunktion `decltype()` statt Traits

```
template<typename Iter, typename Val>
Iter search(Iter p, Iter q, Val val) {
    auto x = *p; // wenn wir den Typ von *p nicht benennen müssen
    auto m = q-p; // wenn wir den Typ von q-p nicht benennen müssen
    using value_type = decltype(*p); // wenn wir Typ von *p benennen
    using Diff_type = decltype(q-p); // wenn wir Typ von q-p benennen
    ... }
```

0.5.9 Algorithmen der Standardbibliothek

0.5.9.1 Eigenschaften von Algorithmen der Standardbibliothek

Algorithmen

- Standardalgorithmen auf Containern
- Headerdatei `<algorithm>`, Namespace `std`
  - `https://en.wikipedia.org/wiki/Algorithm_(C%2B%2B)`
  - `http://en.cppreference.com/w/cpp/algorithm`
  - `http://www.cplusplus.com/reference/algorithm/`
- Beispiele: `copy`, `sort`, `find`, `for_each`, `transform`
- Algorithmen sind *nicht* als Memberfunktionen von Containern implementiert, sondern als *globale Funktionen*
  - Verletzung des objektorientierten Programmiermodells!

© R.-R. Grigat 2003-2019

- Algorithmen
  - verwenden *Iteratoren* für Quellsequenz und Ziel
  - arbeiten auf halboffenen Quellsequenzen
  - können auf *selbstdefinierten* Container-Klassen arbeiten
  - müssen dazu Iteratorklassen für ihre Daten bereitstellen

Beispiele für Algorithmen

```
1  vector<int> vec;
2  vector<int> vec2;
3  vector<int>::iterator pos;
4
5  /* ... */
6  /*Elemente in vec eingefuegen*/
7
8  //Suchen von Minimum und Maximum
9  pos =
10   min_element(vec.begin(); vec.end());
11
12 //Suchen eines Wertes
13 pos = find(vec.begin(), vec.end(), 2);
14
15 //Sortieren
16 sort(vec.begin(), vec.end());
17
18 // Kopieren
19 copy(vec.begin(), vec.end(), vec2.begin())
```

- *min_element (Zeile 10)*
  - Start bei Iterator, der durch `vec.begin()` als erster Parameter erzeugt wird
  - Inkrementiert diesen Iterator, solange die zugehörige Position noch nicht derjenigen des durch `vec.end()` erzeugten zweiten Iterators entspricht
  - Gibt Iterator zurück, der auf das Element mit dem Minimum-Wert zeigt
- *sort (Zeile 16)*
  - Sortiert alle Elemente, die im halboffenen Intervall der durch `vec.begin()` und `vec.end()` erzeugten Iteratoren liegen (1. Parameter inkl., 2. Parameter exkl.)

© R.-R. Grigat 2003-2019

- Sortierung in aufsteigender Reihenfolge

- *copy (Zeile 19)*

  1. Start bei Iterator, der durch `vec.begin()` erzeugt wird *(Quell-Iterator)*
  2. Der Wert wird an die Position des durch `vec2.begin()` erzeugten *Ziel-Iterators* kopiert (3. Parameter)
  3. Anschließend werden Quell- und Ziel-Iterator inkrementiert
  4. Wiederhole 2. und 3., solange Quell-Iterator nicht dem *End-Iterator* (2. Parameter) entspricht

0.5.9.2 Sequenzen (Ranges)

- Jeder Algorithmus arbeitet über halb-offene Sequenzen
- Anfang und Ende der Sequenzen werden übergeben, um nicht ausschließlich auf dem *gesamten* Container arbeiten zu müssen
- Programmierer ist verantwortlich für die *Gültigkeit* des Speicherbereichs der Sequenz

Beispiel

```
1  #include <vector>
2  #include <algorithm>
3
4  int main()
5  {
6    vector<int> vec1;
7    vector<int> vec2;
8    /* ... */
9
10   sort(vec1.begin(), vec1.end());//Gueltige Sequenz
11   sort(vec1.begin(), vec2.end());
12   //Undefiniertes Verhalten, aber kein Compilerfehler!
13 }
```

© R.-R. Grigat 2003-2019

0.5.9.3 Mehrfach-Sequenzen

Einige Algorithmen arbeiten über mehrfache Sequenzen

```
copy(vec1.begin(), vec1.end(), //Quellsequenz, erste Sequenz
     vec2.begin());            //Zielsequenz, zweite Sequenz
```

- Anfang und Ende der ersten Sequenz müssen übergeben werden (als Iterator)
- Für alle anderen Sequenzen muss nur der Anfang übergeben werden (ebenfalls als Iterator)
  - Länge der anderen Sequenzen ergibt sich aus der Länge der ersten Sequenz
- *Wichtig*: Zweite und folgende Sequenzen müssen mindestens so viele Elemente wie erste Sequenz haben, ansonsten Zugriff auf ungültige Speicheradressen
- Zur Vermeidung von Fehlern:
  - Sicherstellen, dass das Ziel genug Elemente hat
  - z. B. *Insert-Iterator* benutzen s. Kapitel0.5.10.1

Beispiel: KP36_4_streamIterators

Beispiel

```
#include <list>
#include <vector>
#include <algorithm>
using namespace std;
int main()
{
  list<int> lst;
  for(int i = 1; i <=5; ++i)
  {
    lst.push_back(i);  // Kein Algorithmus, daher  Aufruf der
  }                    // Memberfunktion direkt möglich
  vector<int> vec;

  copy(lst.begin(), lst.end(), //Quellsequenz
       back_inserter(vec));    //Daten an vec hinten anfügen
}                  // Algorithmus copy benötigt 3 Iteratoren
```

- Alle Elemente aus `lst` werden nach `vec` kopiert
- Elemente werden am Ende von `vec` eingefügt (`back_inserter` ist ein Insert-Iterator, s. u.)

© R.-R. Grigat 2003-2019

0.5.9.4 STL-Algorithmus for_each()

for_each()

- hat Funktion als dritten Parameter
- Funktion wird auf jedes Element einer Sequenz angewandt

Beispiel

```
void print(int value)
{
  cout << value << endl;
}
```

```
vector<int> vec;
/* ... */
for_each(vec.begin(),vec.end(),   // Sequenz
         print);                  // Funktion als Parameter
```

- `print(value)` wird für jedes Element aufgerufen
- also `print(vec[0])`, `print(vec[1])`, ...

Beispiel: KP36_1_forEach_maxElement

0.5.9.5 STL-Algorithmus transform()

transform()

- Funktion wird auf Elemente einer Quellsequenz angewandt
- Ergebnis wird in Zielsequenz abgelegt

Beispiel

```
int square(int value)
{
  return value * value;
}
```

```
vector <int> vec1, vec2;
/* vec1 zunächst mit Werten fuellen (hier weggelassen) */
transform(vec1.begin(), vec1.end(),//Quellsequenz
          back_inserter(vec2),     //Daten an vec2 hinten anfügen
          square);                 //Funktion
```

Ergebnis: `vec2[0] = square(vec1[0])`, `vec2[1] = square(vec1[1])`, ...

© R.-R. Grigat 2003-2019

0.5.9.6 Prädikate in der Standardbibliothek

Definition:
Prädikate sind in der Funktionen, die einen Wert vom Typ bool zurückgeben.

Eigenschaften von Prädikaten:

- werden häufig benutzt, um das Such- oder Sortierkriterium anzugeben
- können unär oder binär sein

0.5.9.7 Unäre Prädikate

Definition:
Unäre Prädikate prüfen eine Eigenschaft eines Arguments.

```
1 bool gt_10(int value)
2 {
3   return (value > 10);
4 }
```

```
1  list<int> lst;
2  list<int>::iterator pos;
3  /* ... */
4  pos = find_if(lst.begin(), lst.end(),   //Sequenz
5                gt_10);                   //Praedikat
```

- `find_if()` wendet `gt_10(int)` auf jedes Element an, solange `gt_10(int)` **`false`** liefert
- sobald `gt_10(int)` **`true`** ergibt, bricht `find_if()` ab und liefert die Position des aktuellen Elements zurück

Beispiel: KP36_07_searchCount

0.5.9.8 Binäre Prädikate

Definition:
Binäre Prädikate vergleichen zwei Argumente.

Eigenschaften:

© R.-R. Grigat 2003-2019

Notwendig, wenn der Operator < nicht für die verwendeten Datentypen definiert ist, aber z. B. sortiert werden soll

Beispiel:

```
class Person {
public:
  string firstname();
  string lastname();
  /*...*/
};

bool compareTwoPersons(const Person& p1, const Person& p2) {
  return (p1.lastname() < p2.lastname()) ||
         ((p1.lastname() == p2.lastname()) &&
          p1.firstname() < p2.firstname());
}
```

```
int main() {
  deque<Person> deq;
  /*...*/
  sort(deq.begin(), deq.end(), compareTwoPersons);
}
```

Beispiel: KP36_09_generateRemove, KP36_10_mutations, KP36_11_sort

0.5.10 Iterator-Adapter

Iterator-Adapter:

- „Gewöhnliche“ Iteratoren sind reine Abstraktionen und stellen die Operatoren `++`, `--`, `==`, `!=`, `*`, `=` bereit
- Iterator-*Adapter* sind dagegen Klassen mit dem Interface von Iteratoren, aber stark *abweichendem Verhalten*
- Beispiele für Iterator-Adapter der Standardbibliothek:
  - Insert-Iteratoren
  - Stream-Iteratoren
  - Reverse-Iteratoren

© R.-R. Grigat 2003-2019

0.5.10.1 Iterator-Adapter: Insert-Iteratoren

Insert-Iteratoren

- Zuweisung eines Wertes an den Insert-Iterator überschreibt den Zielwert nicht, sondern fügt neues Element ein
- Ermöglicht Algorithmen im Insertmodus statt im Überschreibemodus
- Anwendung insbesondere bei Algorithmen, die auf Mehrfachsequenzen arbeiten, z. B. `copy`, `transform`
  - Elemente eines Containers `A` sollen in Container `B` kopiert werden
  - Dabei sollen alte Werte in `B` nicht überschrieben werden
  - Die Elemente sollen in `B` zusätzlich eingefügt werden
  - Verhindert, dass Zielsequenz nicht genügend Elemente enthält, und somit auf ungültige Speicherbereiche zugegriffen wird

0.5.10.2 Iterator-Adapter: Insert-Iteratoren (2)

Arten von Insert-Iteratoren

- Es gibt drei verschiedene Arten von Insert-Iteratoren:
  - `front_inserter iter(cont);`
    `iter = value;` fügt Element `value` am Anfang des Containers `cont` mittels `cont.push_front(value)` ein
  - `back_inserter iter(cont);`
    `iter = value;` fügt Element `value` am Ende des Containers `cont` mittels `cont.push_back(value)` ein
  - `inserter iter(cont, pos);`
    `iter = value;` fügt Element `value` an Position des Iterators `pos` des Containers `cont` mittels `cont.insert(pos, value)` ein
- Container müssen die benötigten Memberfunktionen `push_front`, `push_back` bzw. `insert` enthalten, sonst ist das Verhalten undefiniert
- Direktübergabe obiger Memberfunktionen an Algorithmus unzulässig, Algorithmus-Signatur enthält *Iteratoren*

© R.-R. Grigat 2003-2019

0.5.10.3 Iterator-Adapter: Insert-Iteratoren (3)

Operatorverhalten bei Insert-Iteratoren

- Die Operatoren ++, --, * (Inhaltsoperator) existieren zwar, haben aber für Insert-Iteratoren keine Funktion (bzw. liefern genau genommen `*this`)
- Die Zuweisungsoperation `iter = value`
  - fügt den Wert `value` in den Container ein, dem `iter` zugeordnet ist
  - an die Position, die über die Art des Insert-Iterators `iter` spezifiziert ist
    - also am Anfang bei `front_inserter`
    - an einer mittels Iterator gegebenen Position bei `inserter`

Beispiel

```
1  #include <list>
2  #include <algorithm>
3  using namespace std;
4  int main()
5  {
6          list<int> lst;
7  /* ... */
8          copy (lst.begin(), lst.end(), // Quellsequenz
9                front_inserter(lst))      // Ziel
10 }  // Alle Argumente von copy müssen Iteratoren sein!
```

- copy ist Algorithmus, der Sequenz kopiert
- `front_inserter` ist Iterator für Zielsequenz
  - Gibt an, dass jedes Element von `lst` an den Beginn von `lst` kopiert werden soll
  - Die Liste 1 2 3 4 5 wird zu 5 4 3 2 1 1 2 3 4 5
  - Die Liste „wächst", d. h. Speicherplatz für zusätzliche Elemente durch internen Aufruf von `push_front` reserviert

© R.-R. Grigat 2003-2019

0.5.10.4 Iterator-Adapter: Stream-Iteratoren

Stream-Iteratoren

- *Istream-Iteratoren*: von einem Stream lesen
  - Beispiel: `istream_iterator<int> intReader(cin);`
  - liest Integer-Werte von der Standardeingabe
- *Ostream-Iteratoren*: in einen Stream schreiben
  - Beispiel: `ostream_iterator<int> intWriter(cout);`
  - schreibt Integer-Werte an die Standardausgabe

End-Of-Stream-Iterator

- Wird erzeugt durch parameterlosen Konstruktor eines *Istream*-Iterators
  - Beispiel: `istream_iterator<int> myEOF;`
- Jeder Istream-Iterator wird zu End-Of-Stream Iterator nach ungültiger Eingabe
  - Beispiel: `intReader` wird zu End-Of-Stream Iterator nach Eingabe eines Buchstabens
- geeignet, um ungültige Eingaben abzufangen

Beispiel

```
1  istream_iterator<int> intReader(cin);
2  ostream_iterator<int> intWriter(cout, ", "); // ostream_iterator mit
3                                               // Komma als Trennzeichen
4  istream_iterator<int> intReaderEOF;  //Erzeugen eines
5                                       //End-Of-Stream-Iterators
6                                       //mittels Default-Konstruktor
7  while(intReader!=intReaderEOF)
8  {
9    *intWriter = (*intReader)*2;
10   ++intReader;
11   ++intWriter;
12 }
```

- Jede eingegebene **int**-Zahl wird verdoppelt ausgegeben

© R.-R. Grigat 2003-2019

- Dabei wird Komma (,) zwischen die Werte gesetzt
- Abbruch der **while**-Schleife nach Eingabe eines ungültigen Wertes

Beispiel: KP36_4_streamIterators

0.5.10.5 Iterator-Adapter: Reverse-Iteratoren

Eigenschaften von Reverse-Iteratoren

- Reverse-Iteratoren vertauschen intern die Bedeutung von ++ und --
- Container liefern über Memberfunktionen `rbegin()` und `rend()` Reverse-Iteratoren
- Ermöglicht, dass Algorithmen „rückwärts“ arbeiten

Beispiel

```
1 list<int> lst;
2 /*  ... */
3 copy(lst.rbegin(), lst.rend(),                  // Quelle
4      ostream_iterator<int>(cout, " <-- "));     // Ziel
```

- Kopiert Elemente von `lst` in umgekehrter Reihenfolge
- hier: Ziel ist `ostream_iterator`, mit `cout` als Ausgabestream und <-- als Trennzeichen

Beispiel: KP36_5_inserter, KP36_6_findElem

0.5.11 Funktionsobjekte (Funktoren)

Definition Funktionsobjekt:

- Objekt, das wie eine Funktion aufgerufen werden kann
- Operator () wird dazu überladen
- wird auch funktionsartiges Objekt oder *Funktor* genannt
- kann als Parameter übergeben werden, wenn Funktionsname erwartet wird, z. B. in Algorithmus `for_each()`

© R.-R. Grigat 2003-2019

0.5.11.1 Beispiel: Addierer als Funktionsobjekt

Beispiel

```
template <typename T>
class Addierer
{
  public:
    Addierer (T summand);
    void operator() (T& param);
  private:
    const T m_summand;
};

template <typename T>
Addierer<T>::Addierer (T summand):
  m_summand(summand)
{}
template <typename T>
void Addierer<T>::operator()(T& param)
{
  param += m_summand;
}
```

```
//main.cpp
/* ... */
  Addierer<int> plusDrei(3);
  int a = 4;
  plusDrei(a);
    //bewirkt Aufruf von
    //plusDrei.operator()(a);
  cout << a << endl;
```

- `plusDrei(a)` sieht nur so aus wie Funktionsaufruf
- in Wirklichkeit Aufruf von `plusDrei.operator()(a)`
- Ausgabe: 7

© R.-R. Grigat 2003-2019

0.5.11.2 Funktionsobjekte als Parameter von STL-Algorithmen

Beispiel

```
1 Addierer<int> plusDrei(3);
2 for_each(vec.begin(), vec.end(),  //Sequenz
3          plusDrei);               //Operation
```

- `for_each` ruft für alle Elemente `plusDrei` auf
- also: `plusDrei(vec[0])`, `plusDrei(vec[1])`, ...

Dasselbe Beispiel kürzer

```
1 for_each(vec.begin(), vec.end(),  //Sequenz
2          Addierer<int>(3));       //Operation
```

- Bei Aufruf von `for_each` wird temporäres Funktionsobjekt vom Typ `Addierer<int>(3)` angelegt
- dieses Funktionsobjekt wird mit jedem Element der Sequenz als Parameter ausgeführt
- Beide Versionen des Beispiels verhalten sich identisch

Weiteres Beispiel:

```
1 for_each(vec.begin(), vec.end(), Addierer<int>(*vec.begin()));
```

- `*vec.begin()` ist Inhalt des ersten Elements von vec
- `for_each` erzeugt bei Aufruf temporäres Objekt vom Typ `Addierer<int>`
- Dem Konstruktor `Addierer<int>(*vec.begin())` wird der Inhalt des ersten Elements von vec als Parameter übergeben
- `for_each` addiert also erstes Element von vec zu allen Elementen von vec hinzu

Beispiel: KP36_2_functor

Vorteile von Funktionsobjekten

- Funktionsobjekte sind mächtiger als Funktionen

© R.-R. Grigat 2003-2019

- Funktionsobjekte können Memberfunktionen und -variablen besitzen
  - Funktionsobjekte können dadurch Zustände einnehmen und ändern zur Laufzeit
  - Verschiedene Instanzen von Funktionsobjekten können gleichzeitig verschiedene Zustände haben
  - dadurch unterschiedliches Verhalten verschiedener Funktionsobjekte trotz gleicher Signatur
    - nicht möglich bei Funktionen
    - Verhalten von Funktionen nur variabel durch übergebene Parameter

- Der Algorithmus `for_each` ruft das übergebene Funktionsobjekt `obj` für eine Sequenz auf

Definition:

```
1 template<typename T, typename F> F for_each (T anf, T end, F obj)
2 {
3   while (anf != end) obj(*anf++);
4   return obj;
5 }
```

- T ist Template-Parameter für einen Iteratortyp
- F ist Template-Parameter für einen Funktionsobjekttyp
- `for_each` ruft `obj(*anf)` und folgende auf
- Übergabe des Funktionsobjekts `obj` als Kopie (*by value*),
- Rückgabewert von `for_each` ist die interne Kopie von `obj`
- Der Algorithmus `for_each` kann auch zum Aufruf von Funktionen genutzt werden, die nicht im **`operator()`** eines Objekts gekapselt sind.

Beispiel:

```
1 template<typename T> void gibAus(T t) {std::cout << t << std::endl;}
2 int main (int argc, char* argv[])
3 {
4   vector<unsigned short> vec;
5   vec.push_back(3);
6   vec.push_back(5);
7   for_each(vec.begin(), vec.end(), gibAus<unsigned short>);
8 }
```

Bsp: Funktn double fnct (double) cast

© R.-R. Grigat 2003-2019

static_cast< double(*)(double) > (gibAus);

0.5.11.3 Beispiel: Summenbildung mit Funktionsobjekten

Beispiel

```
1  //summe.hpp
2  template<typename T> class Summe
3  {
4    public:
5      Summe();
6      void operator() (T& x);
7      T ergebnis() const;
8    private:
9    T m_ergebnis;
10 };
11
12 template<typename T>
13 Summe<T>::Summe():
14   m_ergebnis(0)
15 {}
```

```
17 template<typename T>
18 void Summe<T>::operator() (T& x)
19 {
20   m_ergebnis += x;
21 }
22
23 template<typename T>
24 T Summe<T>::ergebnis() const
25 {
26   return m_ergebnis;
27 }
```

```
1  //main.cpp
2  /* ... */
3  Summe<double> s1, s2, s3;  // 3 Funktoren
4  s2=for_each(vec1.begin(), vec1.end(),s1);
5  s3=for_each(vec2.begin(), vec2.end(),s1);
6  // Call by Value, s1 bleibt unverändert
7  // for_each liefert Kopien als s2 und s3
```

```
1  //main.cpp
2  /* ... */
```

© R.-R. Grigat 2003-2019

```
3 Summe<double> s1, s2, s3;    // 3 Funktoren
4 s2 = for_each(vec1.begin(), vec1.end(), s1);
5 s3 = for_each(vec2.begin(), vec2.end(), s1);
```

- s1, s2 und s3 werden im Konstruktor mit 0 initialisiert
- Zeile 4 bewirkt scheinbar s1(vec1[0]), s1(vec1[1]), ...
- Zeile 5 bewirkt scheinbar s1(vec2[0]), s1(vec2[1]), ...
- for_each ruft aber lokale Kopie von s1 auf (*by value*), Rückgabe von for_each sind die Kopien, s1 unverändert
- s1, s2 und s3 besitzen eigene (unterschiedliche) Zustände
- Realisierung mit einer einfachen Funktion nicht möglich

0.5.12 Multithreading

Multithreading

- Ein Thread (Faden) ist ein sequentiell ausgeführter Programmabschnitt.
- Jeder Thread hat eigenen Stack (lokale Daten).
- Threads können sich Adressraum teilen (globale Daten), Synchronisierung konkurrierender Speicherzugriffe nötig.
- Nur ein Thread pro Prozessor zur Zeit ausführbar.
- Zeitsparend auf Mehrprozessorsystemen, z. B. vier parallele Threads auf Quad-Core Prozessor.
  - Verschieden schnelle unabhängige Threads blockieren sich nicht (z. B. Berechnung läuft während langsamer Eingabe).
  - Beschleunigung rechenintensiver Programme durch Parallelisierung.

Threads erzeugen

- Threads sind Objekte der Klasse thread.
- Zustände solcher Objekte sind *lauffähig*, *aktiv*, *blockiert* oder *beendet*.

© R.-R. Grigat 2003-2019

- Beendetes Objekt stellt keinen Thread mehr dar.
- Beispiel: Konstruktoraufrufe von Threads

```
1 #include <thread>
2 void main(){
3   thread t1(f1);          // Globale Funktion f1() ohne Parameter
4   thread t2(f2, 5, 17);   // Globale Funktion f2(5, 17)
5   thread t3;              // t3 stellt noch keinen Thread dar
6   t1.join();              // Warten, bis t1 beendet
7   t2.join();              // Warten, bis t2 beendet
8                           // t1, t2 stellen nun keinen Thread mehr dar
9 }
```

Beispiel: KP35_1_thread_sort

Kopieren, Zuweisung und Move von Threads

- Zwei Objekte der Klasse `thread` können nicht denselben Zustand darstellen.
- Kopierkonstruktur und Zuweisung sind daher unterbunden.

```
1 thread(const thread&) = delete;
2 thread& operator=(const thread&) = delete;
```

- Move-Konstruktor und Move-Zuweisung erlaubt

```
1 t1 = thread t(f);   // Funktion f() global, t(f) temporär
```

- Thread kann mit Funktionsobjekt initialisiert werden, arbeitet intern stets mit *Kopie* des Funktionsobjekts.

Beispiel: KP35_2_thread_initialize

Konkurrierende Speicherzugriffe

- Zeitgleiche Speicherzugriffe verschiedener Threads (racing conditions) können zu Inkonsistenz führen.
- `mutex` weist einem Thread ein Betriebsmittel exklusiv zu, bis dieser Thread es wieder freigibt.
- Versuchen andere Threads in den Besitz des `mutex` zu kommen, werden sie blockiert bis zur Freigabe des `mutex`.
- Sperren mit `lock()`, Freigabe mit `unlock()`

© R.-R. Grigat 2003-2019

Mutex Beispiel

```
#include <thread>
#include <mutex>
#include <string>
#include <iostream>

std::string s;   // Speicher für konkurrierende Zugriffe
std::mutex mtx;  // Mutex global vereinbaren

void writeHallo(){
    mtx.lock();
    s = "Hallo"; std::cout << s ;
    mtx.unlock();
}
int main(){
    std::thread t1(writeHallo);
    std::thread t2(writeHallo); // Ohne Mutex HaHllalolo möglich
    t1.join(); t2.join();       // Mit Mutex HalloHallo
    return 0;
}
```

Beispiel: KP35_3_mutex

Thread-ID

- Jedem Objekt der Klasse `thread` ist ein eindeutiger Wert zugeordnet, die Thread-ID.
- Die Thread-ID ist ein Objekt der Klasse `id`, einer lokalen Klasse `thread::id` in `thread`.
- Thread-ID ist positive Ganzzahl (Thread-Objekt ist mit Thread verbunden) oder `Not-any-thread` (Thread-Objekt mit keinem Thread verbunden).
- Ermitteln der Thread-ID von Thread t mit `t.get_id();`, `Not-any-thread` liefert Wert 0.
- Ermitteln der Thread-ID aus der Funktion f in Thread t(f) heraus über globale Funktion `t.get_id();` im Namensbereich `this_thread`.

```
thread::id x = this_thread::get_id();  // in Funktion f im thread t
```

Detach

© R.-R. Grigat 2003-2019

- Verbindung eines Thread-Objekts zum Thread vorzeitig lösen mit `t.detach();`
- `t.get_id()` liefert nun Pseudo-Wert `Not-any-thread`.
- Thread wird bis zu seiner Beendigung weiter ausgeführt.
- Thread selbst behält seine ID, die t vorher zugeordnet war. Diese wird im aktiven Thread von `this_thread::get_id();` geliefert.

Beispiel: KP35_4_threadID

Deadlocks

- Wenn Methode `unlock()` nicht aufgerufen wird hängt das Programm, sobald ein weiterer `lock()` auf denselben Mutex erfolgt. Abhilfe durch Locks, also Objekte der Template-Klasse `lock_guard<mutex>`.
- Konstruktor ruft `lock()` auf, Destruktor ruft `unlock()` auf.
- Lock wird als lokale Variable in auszuführender Funktion des Threads angelegt, diese wird am Ende des Gültigkeitsbereichs zerstört und ruft dadurch `unlock()` auf.

```
lock_guard<mutex> lock(mtx);
```

Beispiel: KP35_5_lock

Rekursiver Mutex

- Rekursive Routine ruft sich selbst erneut auf und dadurch auch `lock()`. Konsequenz ist ein Deadlock.
- Klasse `recursive_mutex` ermöglicht rekursive Aufrufe in einem Thread, während andere Threads weiterhin blockiert werden.
- Der rekursive Lock `recloc` wird erst dann zerstört, wenn sein Geltungsbereich im ersten Aufruf der auszuführenden Routine (oberste Rekursionsebene) wieder verlassen wird.

```
lock_guard<recursive_mutex> reclock(m);
```

Bedingungsvariablen

© R.-R. Grigat 2003-2019

- Consumer in Zustand *blockiert*, wenn Puffer leer ist.
- Producer weckt Consumer auf, wenn Daten geschrieben.
- Producer in Zustand *blockiert*, wenn Puffer voll ist.
- Synchronisierung über Bedingungsvariablen (Objekte der Klasse `condition_variable`)
- Bedingungsvariablen können Threads blockieren oder Blockierungen aufheben
- `condition_variable` arbeitet mit Lock vom Typ der Template-Klasse `unique_lock<mutex>` zusammen

```
#include <mutex>
#include <condition_variable>
condition_variable cond;
mutex mtx;
unique_lock<mutex> lock(mtx);
cond.wait(lock);     //
cond.notify_one();  // hebt Blockierung des nächsten wartenden Threads auf
```

Beispiel: KP35_6_producer_consumer

Lambda-Funktionen

- Lambda-Ausdrücke (Lambda-Funktionen, umgangssprachlich Lambdas) sind lokale, anonyme *Funktionsobjekte*, die auf Objekte ihrer Umgebung zugreifen können.
- Jeder Lambda-Ausdruck definiert eine anonyme Klasse mit einem eindeutigen Typ (Closure Type) und stellt ein Objekt (Closure-Objekt oder *Closure*) dieser Klasse dar. Der Aufrufoperator **`operator()`** ist dem Lambda-Ausdruck entsprechend überladen.

```
[&](int a){return a<x;}
```

- Lambda-Einführung [&] greift auf lokale Variablen der Umgebung (hier x) über Referenzen & zu. Die Lambda-Einführung enthält die *Aufrufparameter des Konstruktors* des Closure Types.

Syntax der Lambda-Ausdrücke

capture Lambda-Einführung, Aufrufparameter des Konstruktors. [] kein Zugriff auf lokale Umgebung, [=] Zugriff per Kopie, [&] Zugriff per Referenz, [=,&var] alle per Kopie, aber var per Referenz

© R.-R. Grigat 2003-2019

(parameterliste) Deklarationen der Funktionsparameter (falls Parameter vorhanden)

mutable Nicht-Referenzvariablen in Lambda-Einführung per Default const, außer mutable wird angegeben (optional).

noexcept Funktion wirft keine Exception (optional).

->return-typ Ohne nachgestellten Return-Typ wird Return-Typ aus Typ des return-Ausdrucks abgeleitet oder ist void, wenn es keine return-Anweisung gibt

0.5.12.1 Lambda-Einführungen

Lambda-Einführungen mit und ohne Erfassungsliste (EL)

[ ] keine lokalen Umgebungsvariablen erfasst

[EL] explizite Erfassung lokaler Umgebungsvariablen als Wert, bei vorangestelltem & als Referenz, kann this enthalten und Variablen gefolgt von ...

[&] implizite Erfassung aller lokalen Umgebungsvariablen als Referenz

[=] implizite Erfassung aller lokalen Umgebungsvariablen als Wert (Kopie)

[&,EL] implizit als Referenz, explizite Variablen als Wert, kein & vor expliziten Variablen zulässig, kann **this** enthalten (stets als Referenz)

[=,EL] implizit als Wert, & muss vor allen expliziten Namen stehen, darf *nicht* **this** enthalten

0.5.12.2 Lambda-Ausdruck und Funktor

Lambda-Ausdruck (Quelle: Stroustrup)

```
// v[i] an os ausgeben, wenn v[i]%m==0
void print_modulo(const vector<int>& v, ostream& os, int m){
for_each(begin(v),end(v),[&os,m](int x) {if(x%m==0) os<<x<<'\n';} );
}
```

```
// Benannter Lambda-Ausdruck
void print_modulo(const vector<int>& v, ostream& os, int m){
auto Modulo_print = [&os,m](int x) {if(x%m==0) os<<x<<'\n';};
for_each(begin(v),end(v), Modulo_print);   // Benannter Lambda-Ausdruck
}
```

© R.-R. Grigat 2003-2019

```
// Äquivalentes Funktionsobjekt ohne Lambda-Ausdruck
class Modulo_print {
ostream& os; int m;
public:
Modulo_print(ostream& s, int mm) :os(s), m(mm) {}
void operator()(int x) const{ if (x%m==0) os << x << '\n'; }
};
void print_modulo(const vector<int>& v, ostream& os, int m){
for_each(begin(v), end(v), Modulo_print{os,m});  } // Aufruf des Funktors
```

Generische Lambdas

- Als Typ für einen Parameter ist **auto** zulässig.
- Generisches Lambda verhält sich wie eine Template-Funktion.
- Compiler generiert anonyme Klasse, die den Aufrufoperator () als Funktions-Template inline überlädt.

```
auto twice = [](auto a){return a+a; };
cout << twice(7) << endl;      // 14
cout << twice("go") << endl;  // gogo
[](){ cout << "Direkter Aufruf" << endl; } ();
```

- In Zugriffsdeklaration [] können auch neue lokale Variablen und Referenzen deklariert werden, Typ wird vom Initialisierer abgeleitet.

```
int   n = 10;
auto lf = [&n, max=n+90]()->int{...};  // max erhält den Typ int
```

Beispiel: KP36_3_lambda